

CONTENTS

The Learning Framework

Feedforward Neural Networks

Activation Functions

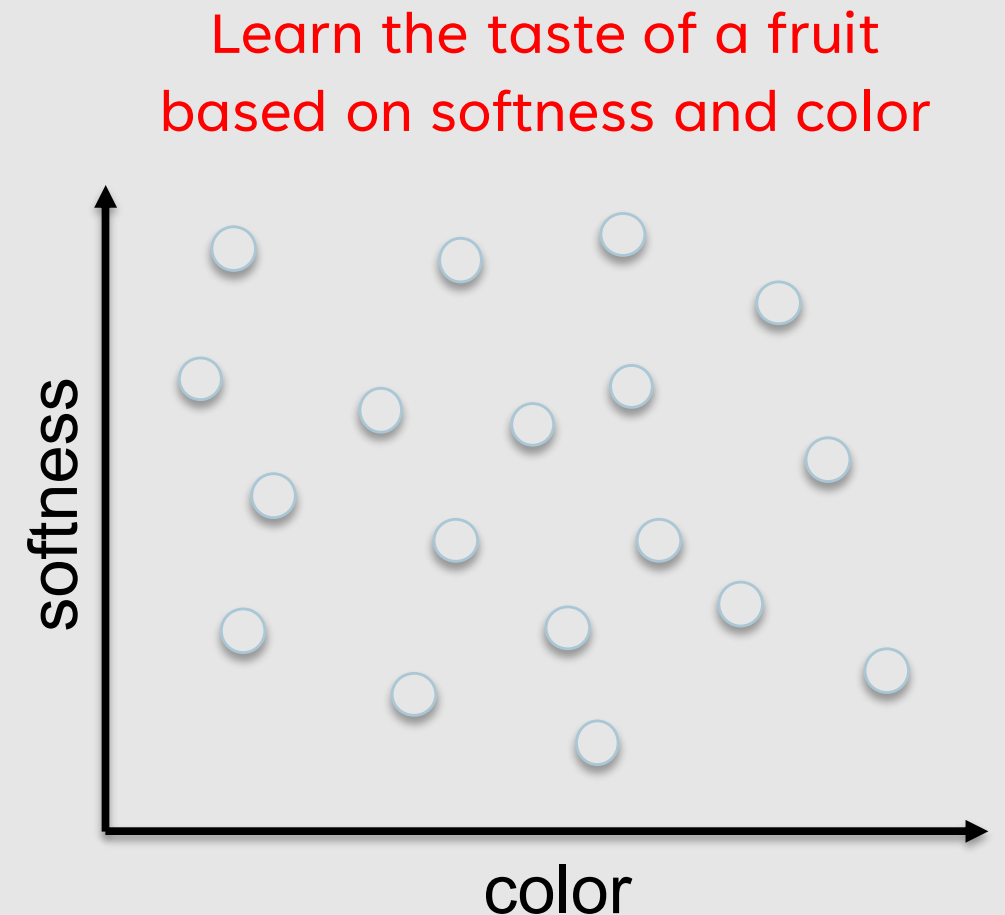
Regularization Techniques

Training Deep Neural Networks

THE LEARNING FRAMEWORK

Learner's input:

- Domain set $\mathcal{X} \rightarrow$ features space
- Label set $\mathcal{Y}$ e.g. $\mathcal{Y} = \{0,1\}$
- Training data $S = \{(x_i, y_i)\}_{i=1}^m$



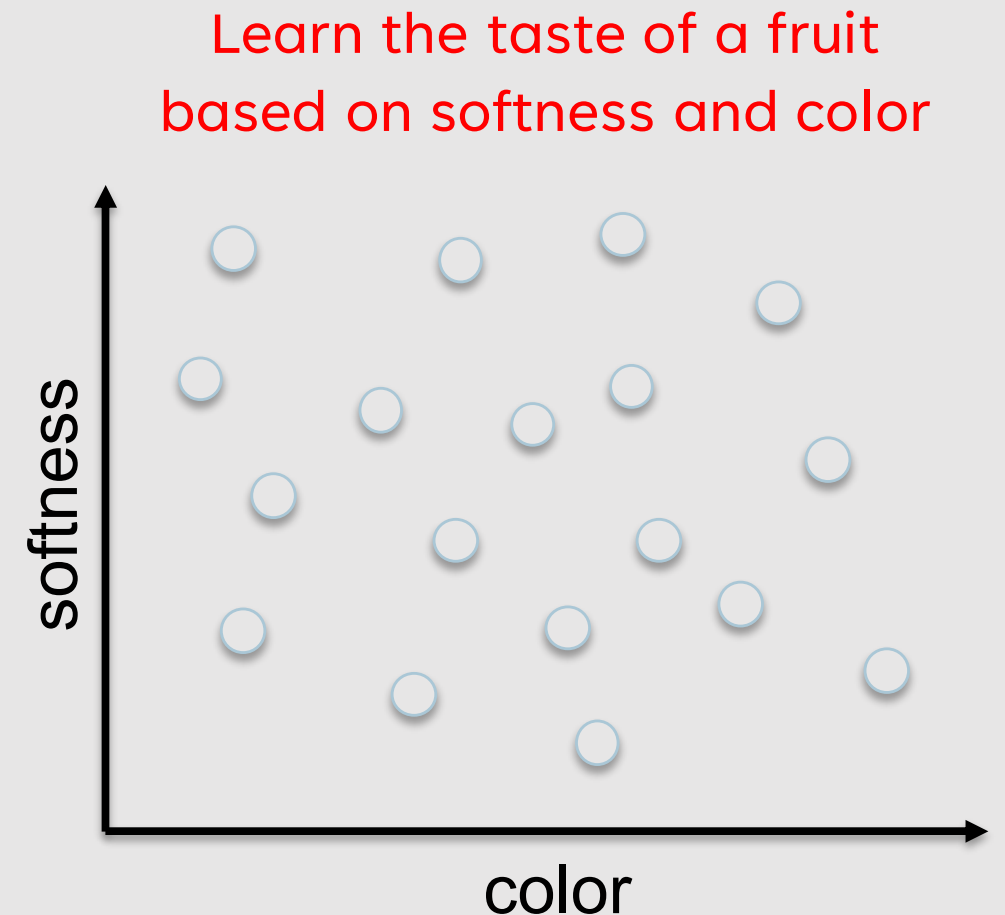
THE LEARNING FRAMEWORK

Learner's input:

- Domain set $\mathcal{X} \rightarrow$ features space
- Label set $\mathcal{Y}$ e.g. $\mathcal{Y} = \{0,1\}$
- Training data $S = \{(x_i, y_i)\}_{i=1}^m$

Learner's output:

- Prediction rule $h \in \mathcal{H}: \mathcal{X} \rightarrow \mathcal{Y}$



THE LEARNING FRAMEWORK

Learner's input:

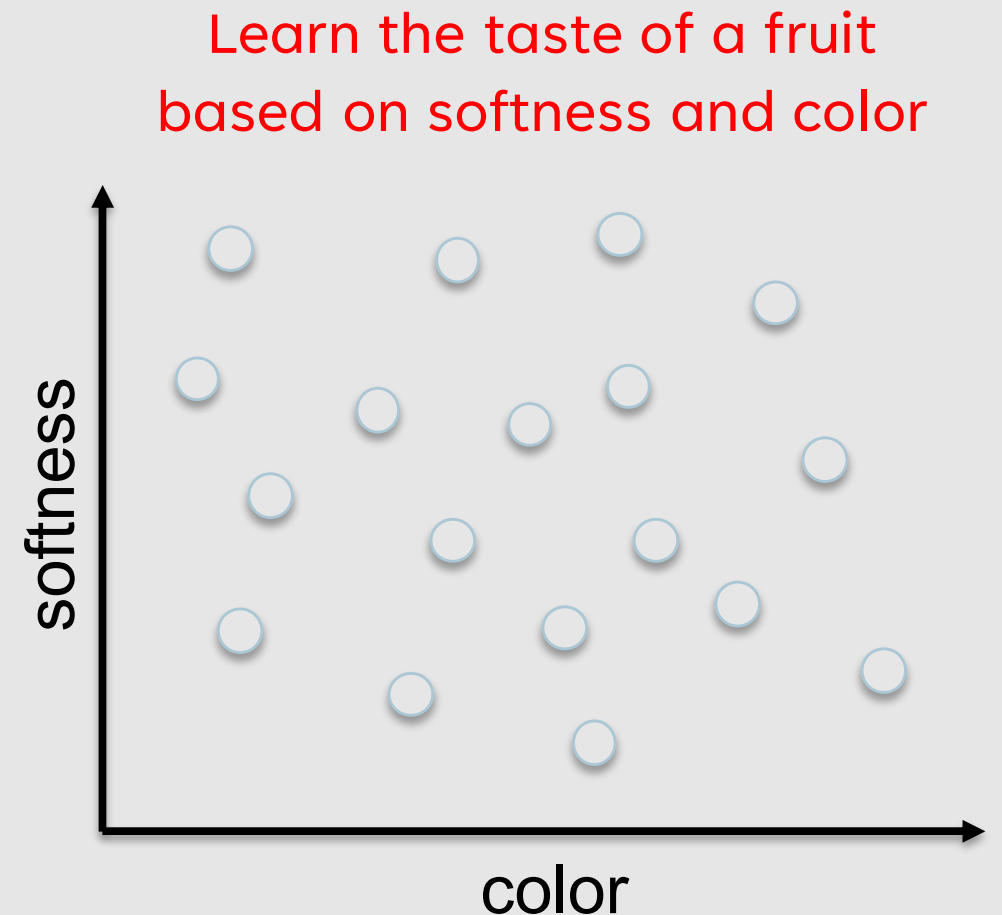
- Domain set $\mathcal{X} \rightarrow$ features space
- Label set $\mathcal{Y}$ e.g. $\mathcal{Y} = \{0,1\}$
- Training data $S = \{(x_i, y_i)\}_{i=1}^m$

Learner's output:

- Prediction rule $h \in \mathcal{H}: \mathcal{X} \rightarrow \mathcal{Y}$

Measures of Success:

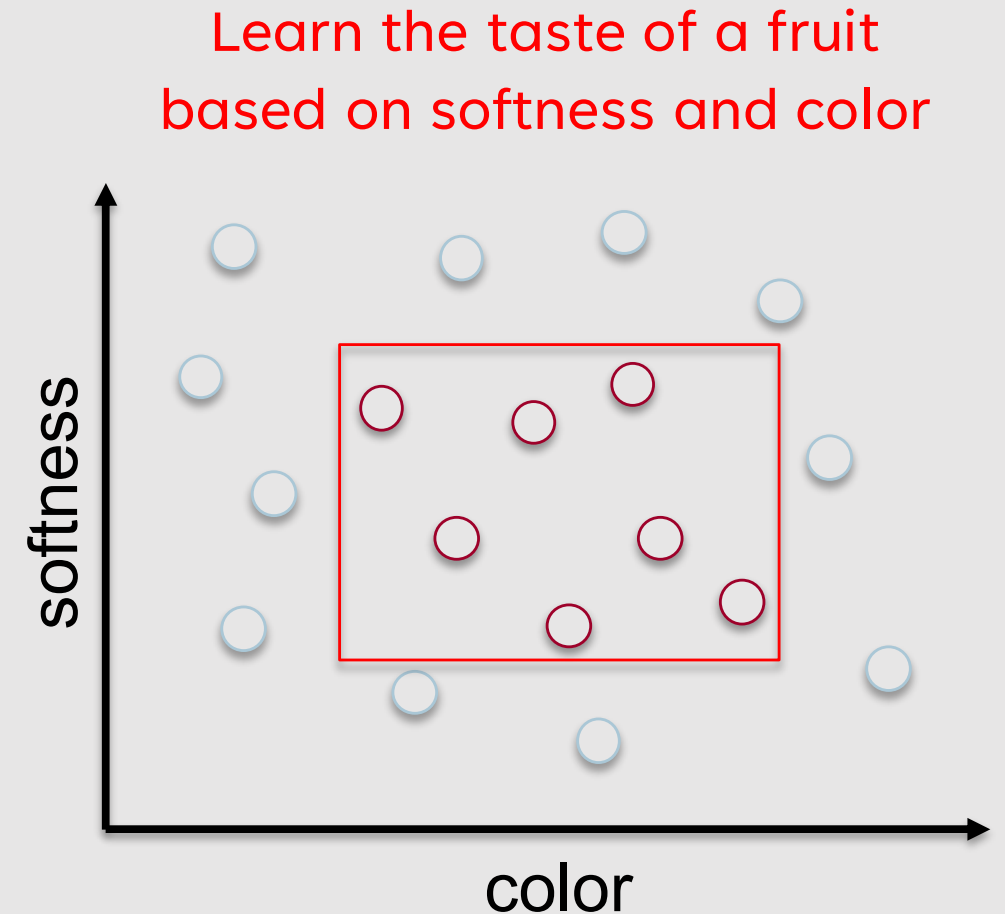
- Classification accuracy, MSE, etc.



THE LEARNING FRAMEWORK

How the prediction rule can be derived?

- Minimize training error



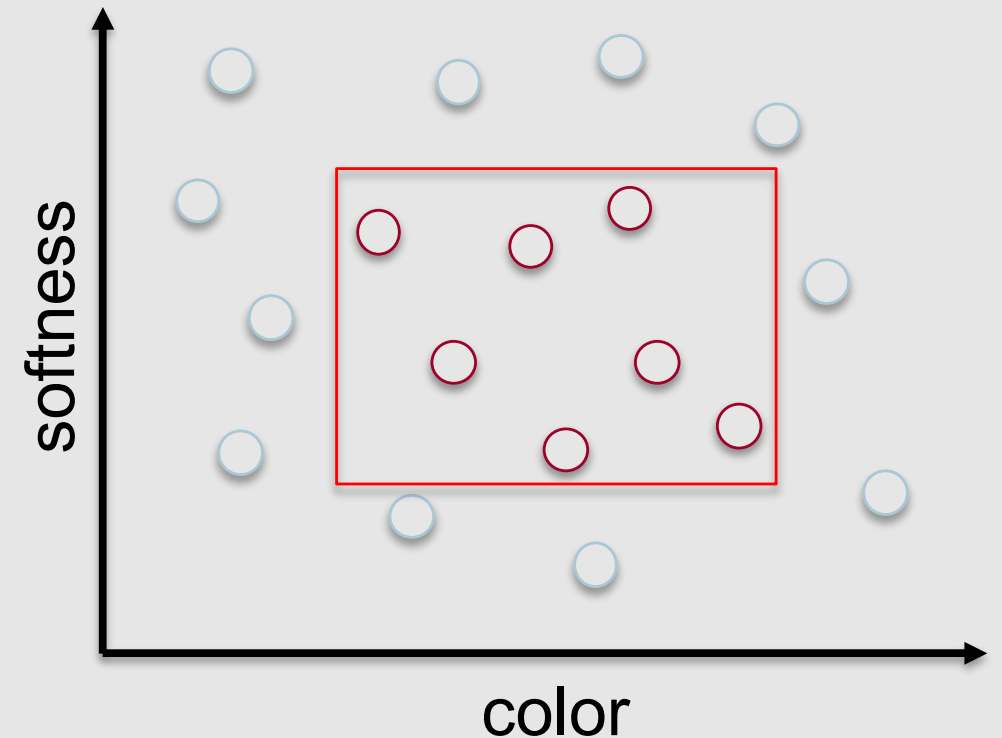
THE LEARNING FRAMEWORK

How the prediction rule can be derived?

- Minimize training error

$$h_S(x) = \begin{cases} y_i & \text{if } \exists i \in [m] \text{ s.t. } x_i = x \\ 0 & \text{otherwise.} \end{cases}$$

Learn the taste of a fruit
based on softness and color



THE LEARNING FRAMEWORK

How the prediction rule can be derived?

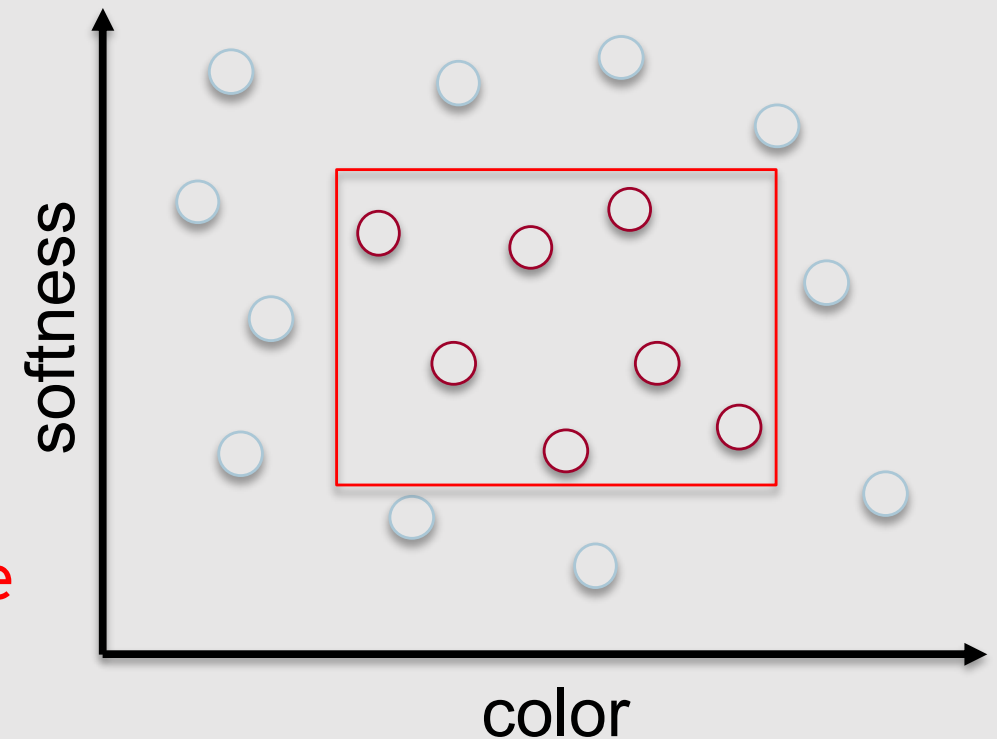
- Minimize training error

$$h_S(x) = \begin{cases} y_i & \text{if } \exists i \in [m] \text{ s.t. } x_i = x \\ 0 & \text{otherwise.} \end{cases}$$

Overfitting

Not every family of functions $\mathcal{H}$ can be
used for learning problems

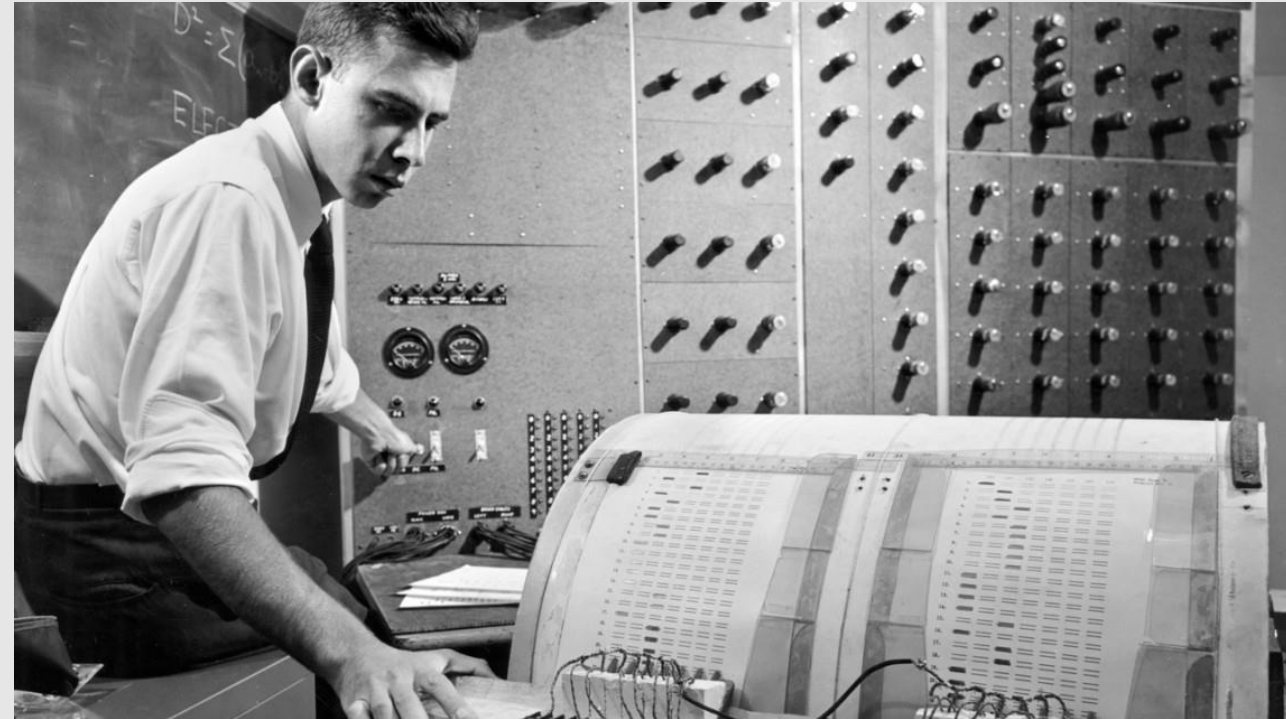
Learn the taste of a fruit
based on softness and color



FEEDFORWARD NETWORKS

Historical Notes

- Perceptron (1943, 1958)
 - Machine and not software
 - Excitement
 - Strong AI

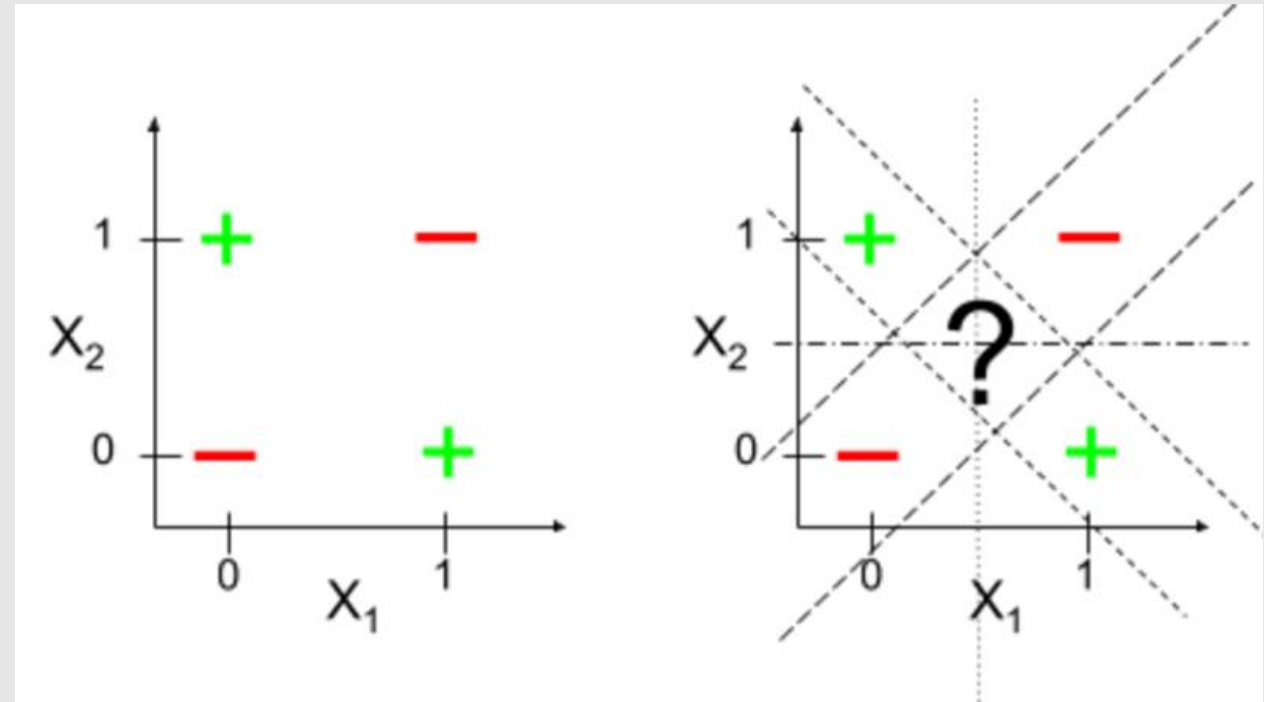


<https://news.cornell.edu/stories/2019/09/professors-perceptron-paved-way-ai-60-years-too-soon>

FEEDFORWARD NETWORKS

Historical Notes

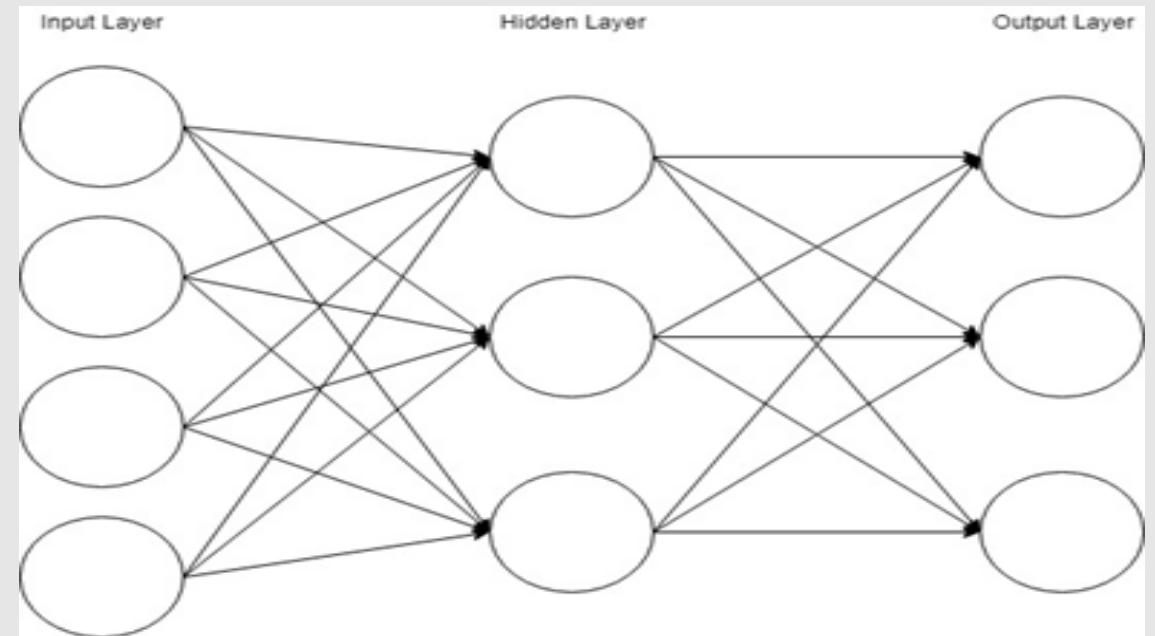
- Minsky & Papert paper (1969)
 - Perceptron limitations
 - Learning XOR
 - Backlash against the entire neural network approach



FEEDFORWARD NETWORKS

Historical Notes

- Multilayer perceptron
 - Nonlinear (XOR solved)
 - Universal approximation properties
 - Backpropagation algorithm (1986)



<https://deeptai.org/machine-learning-glossary-and-terms/multilayer-perceptron>

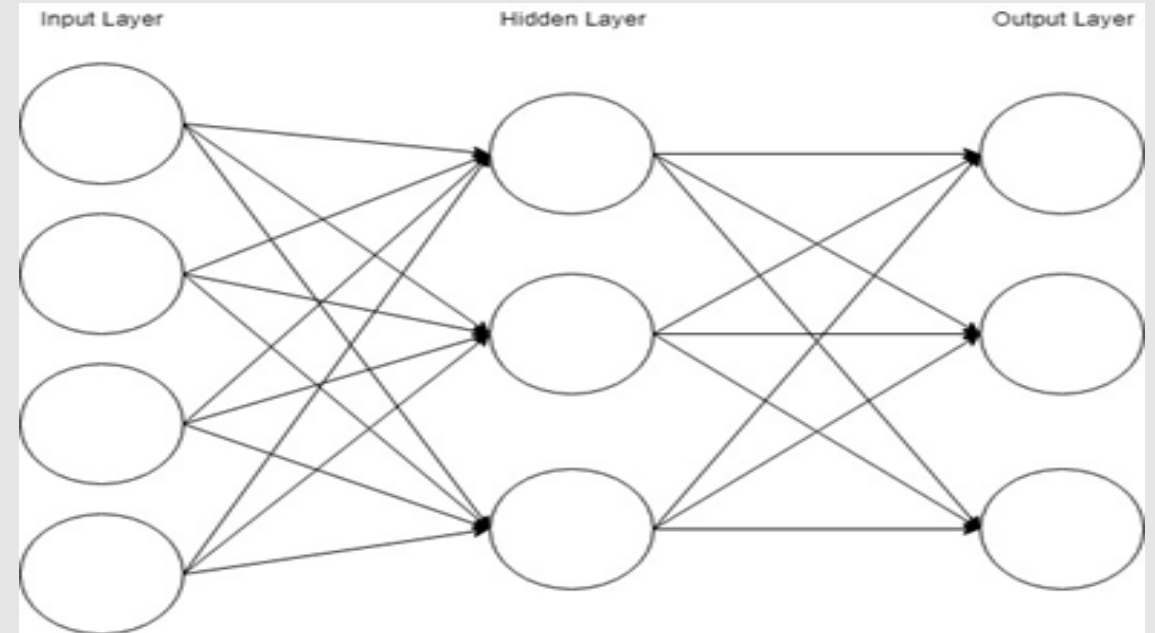
FEEDFORWARD NETWORKS

Historical Notes

- Multilayer perceptron
 - Nonlinear (XOR solved)
 - Universal approximation properties
 - Backpropagation algorithm (1986)

Applicability:

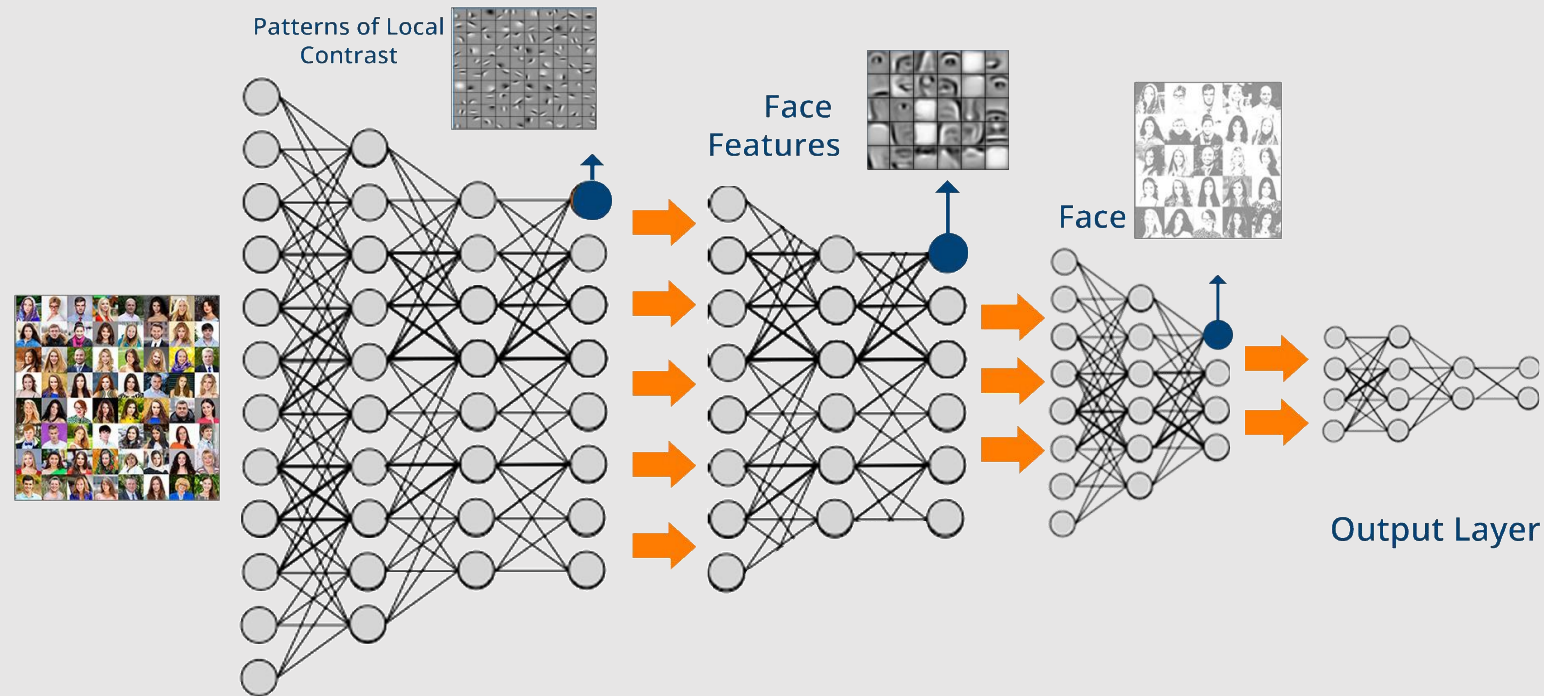
- Computational cost
- Algorithmic Problems



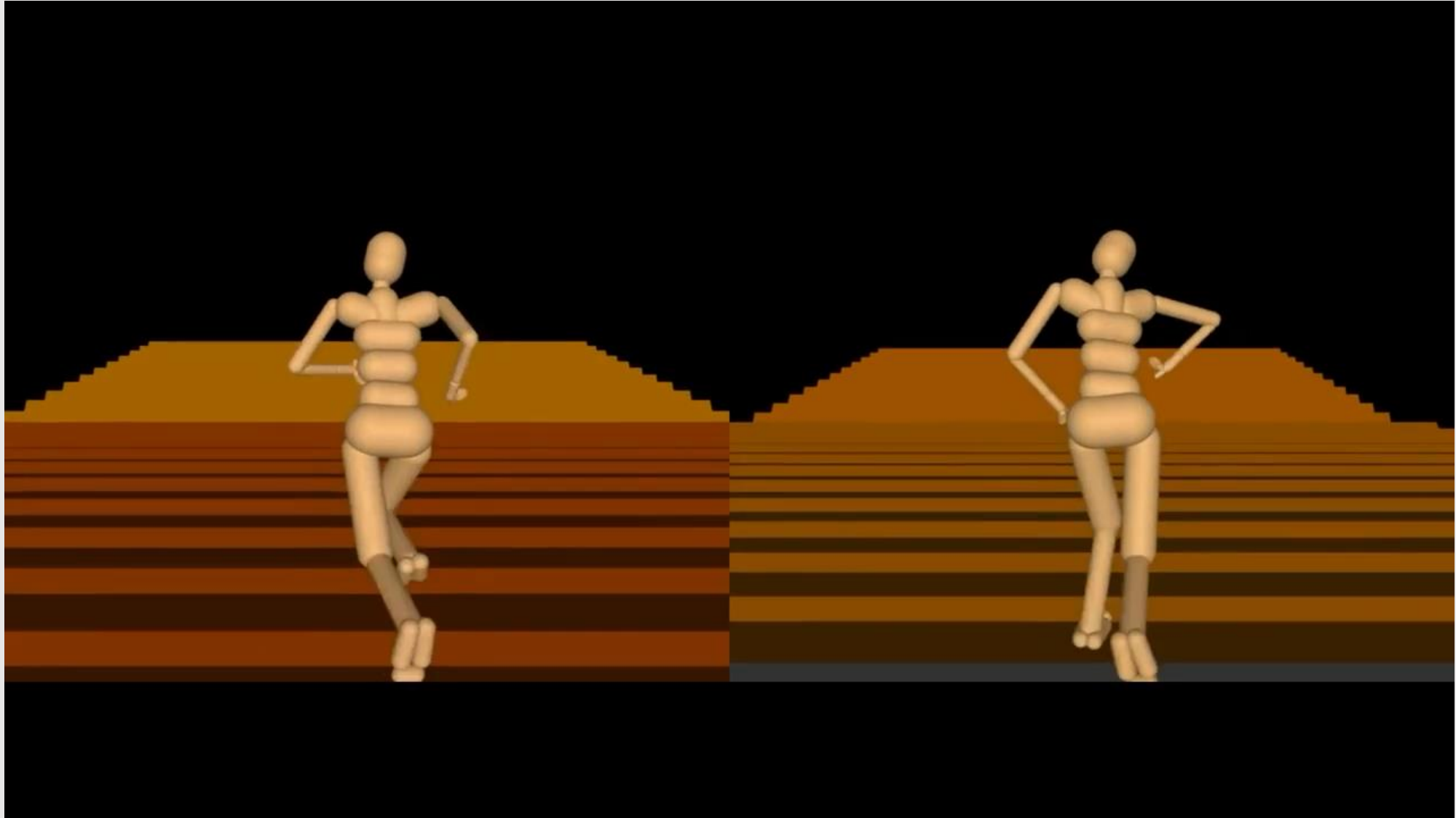
FEEDFORWARD NETWORKS

Historical Notes

- Deep learning
 - Computational power – hardware
 - A family of techniques for solving algorithmic problems
 - State-of-the-art results for many applications



<https://thedata scientist.com/what-deep-learning-is-and-isnt/deep-neural-network-what-is-deep-learning-edureka/>



Merel, Josh, et al. "Learning human behaviors from motion capture by adversarial imitation." arXiv preprint arXiv:1707.02201 (2017).



<https://www.youtube.com/channel/UCOKTr4-UCZAd7Y56bKQWbvQ>

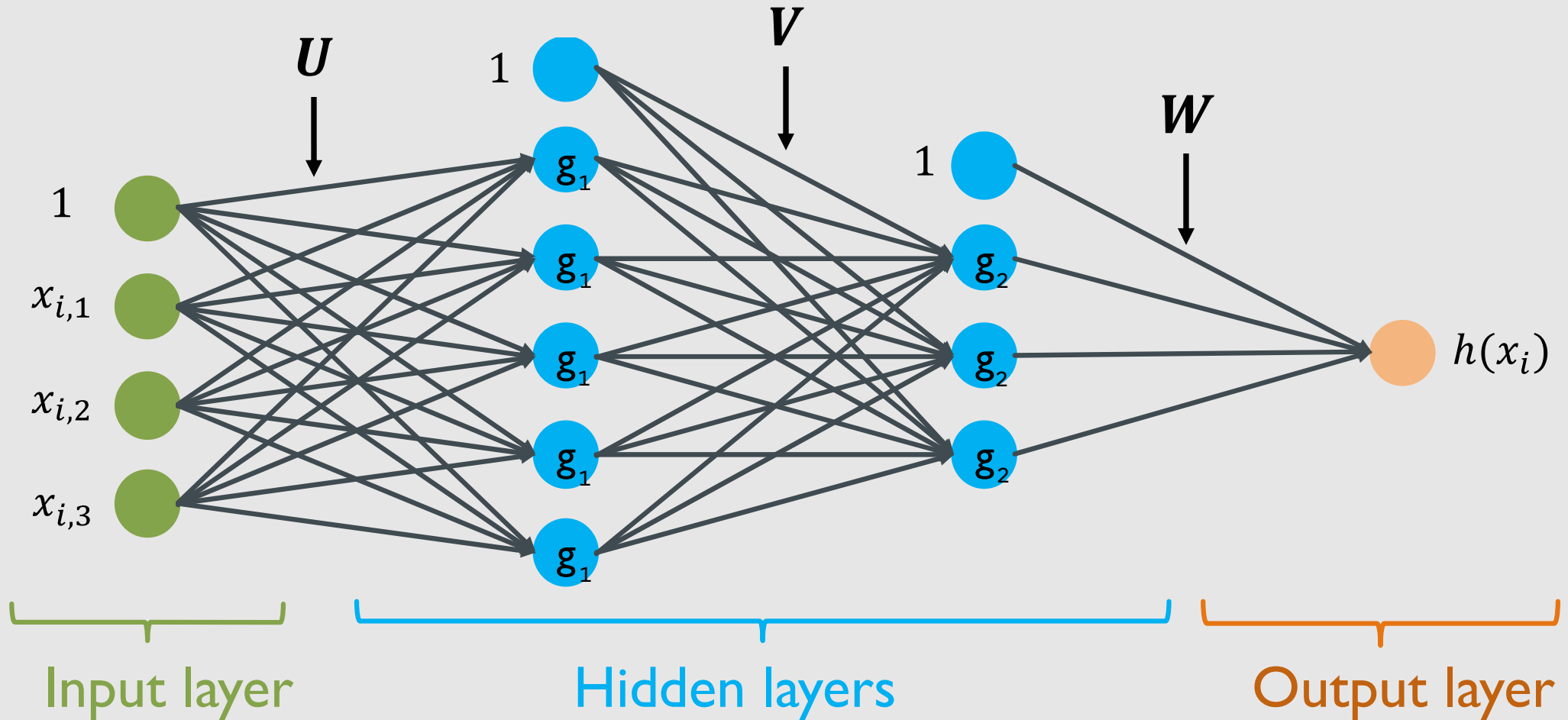






<https://github.com/michalnand/>

FEEDFORWARD NETWORKS



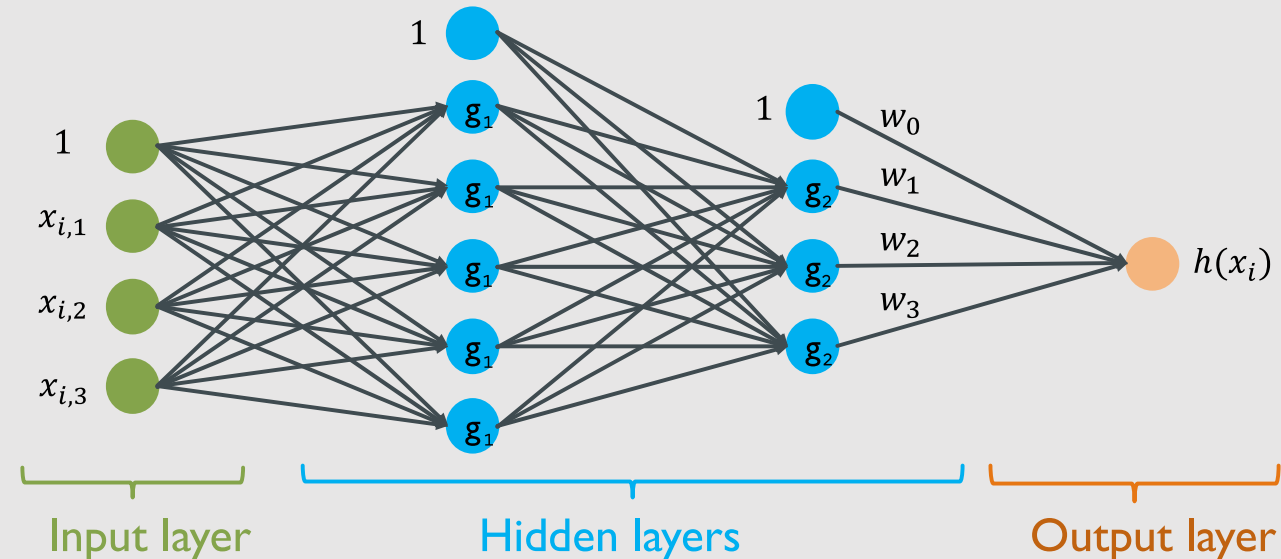
FEEDFORWARD NETWORKS

Output of a network

$$h(x_i) = f([w_1, w_2, w_3]^T g_2(z_1) + w_0)$$

The function f can be:

- the **Identity**,
- the **Logistic**,
- the **Softmax**



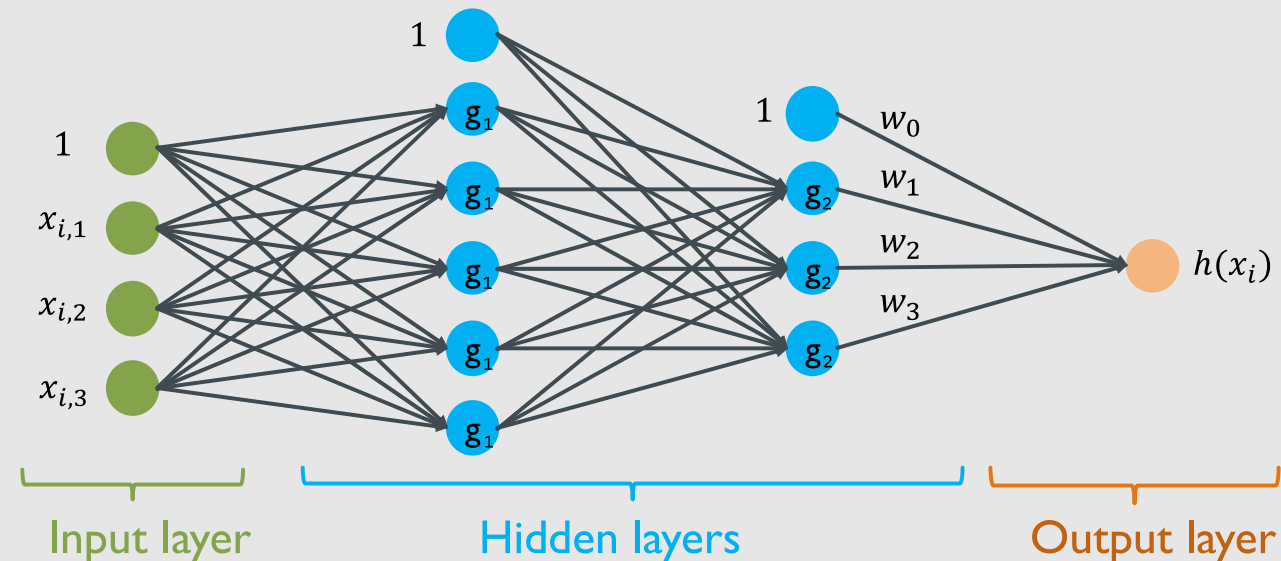
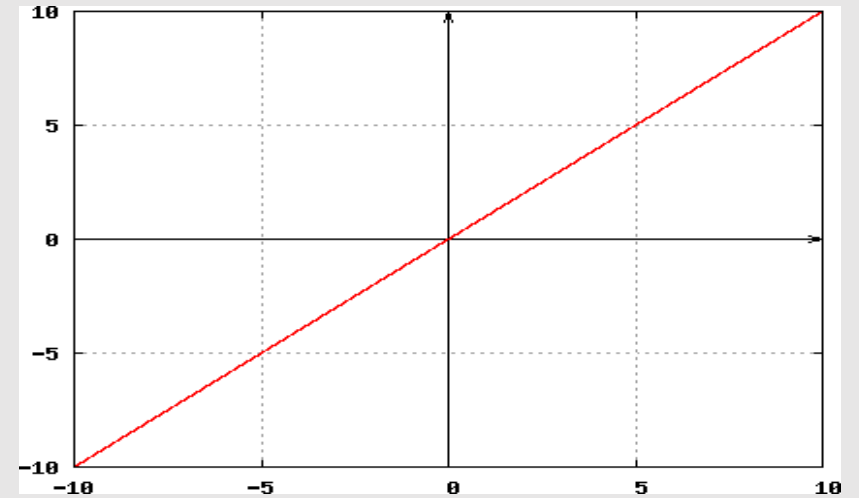
FEEDFORWARD NETWORKS

Output: Identity

$$h(x_i) = [w_1, w_2, w_3]^T g_2(z_1) + w_0$$

The output is a **real** number α_i .

- Regression problems
- $a_i = \mathbb{E}[y_i|x_i]$
- Mean Squared Error



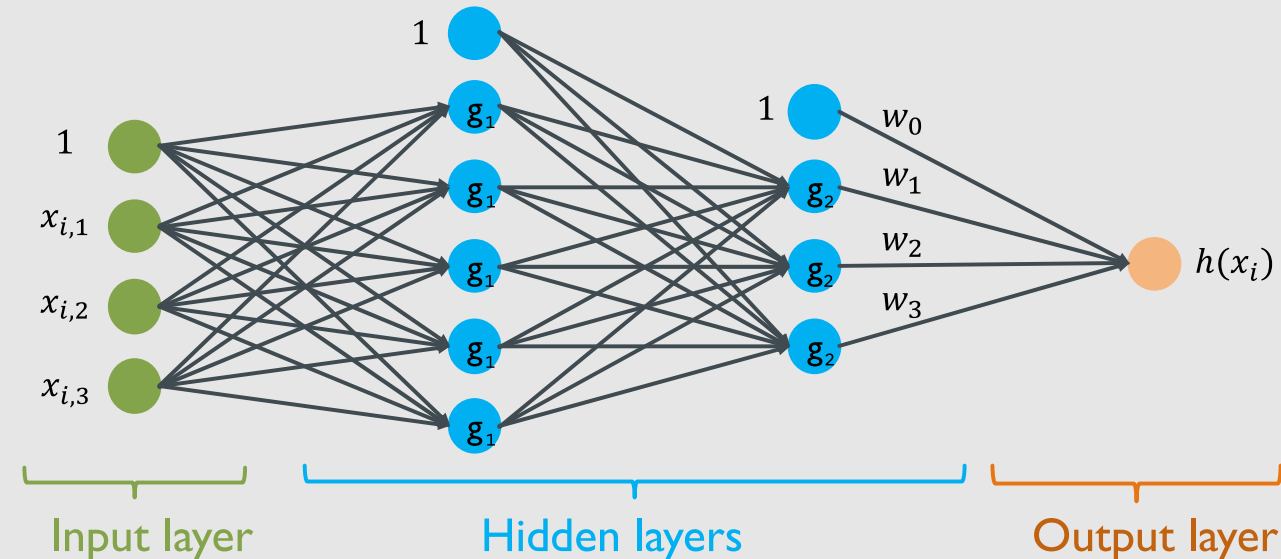
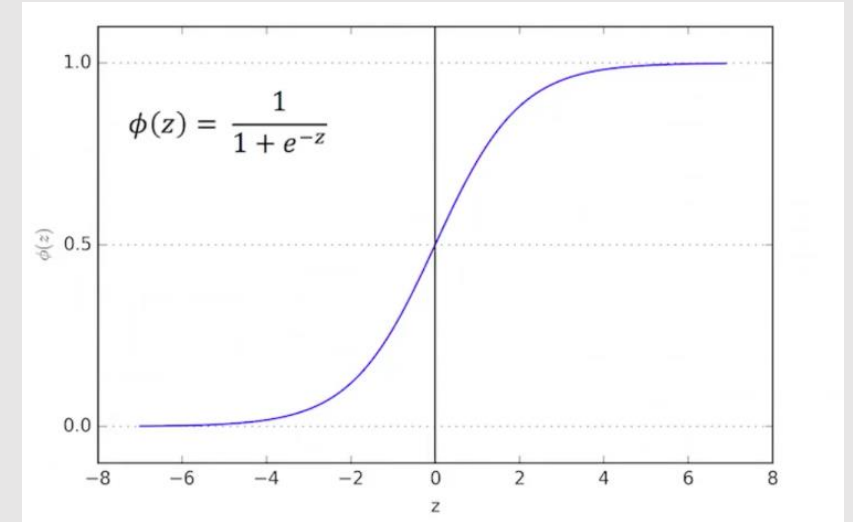
FEEDFORWARD NETWORKS

Output: Logistic

$$h(x_i) = \sigma([w_1, w_2, w_3]^T g_2(z_1) + w_0)$$

The output is a number $\alpha_i \in (0,1)$.

- Binary classification problems
- $a_i = \mathbb{P}[y_i = 1|x_i]$
- Negative log-likelihood



FEEDFORWARD NETWORKS

Output: Logistic

$$h(x_i) = \sigma(\underbrace{[w_1, w_2, w_3]^T g_2(z_1) + w_0}_{\tau(x_i)})$$

The output is a number $\alpha_i \in (0,1)$.

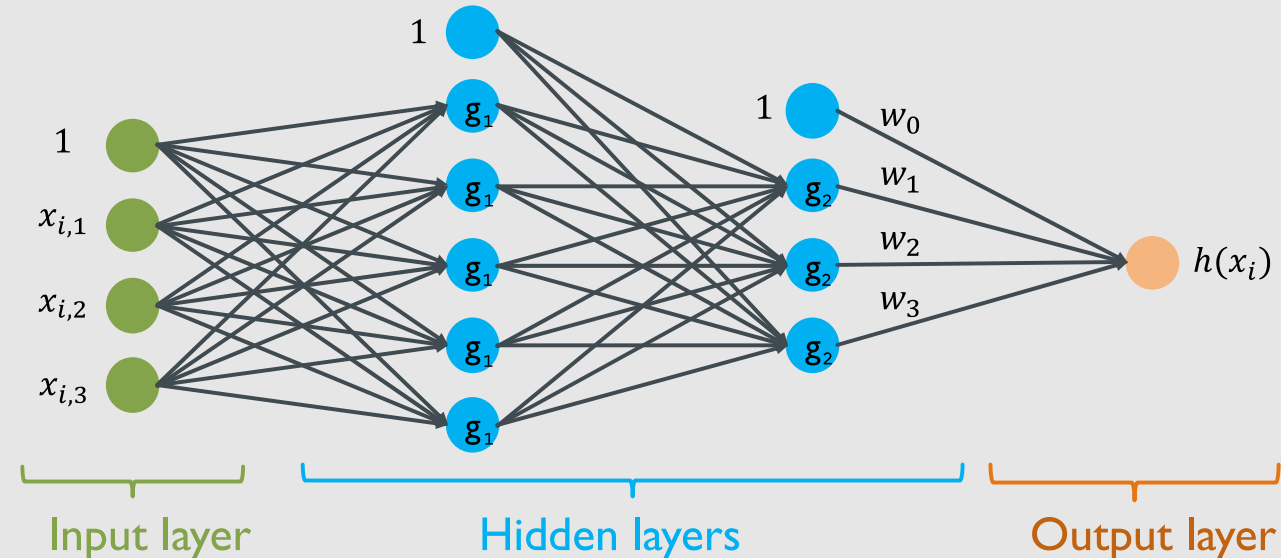
- Binary classification problems
- $a_i = \mathbb{P}[y_i = 1|x_i]$
- Negative log-likelihood

$$\log \tilde{\mathbb{P}}(y_i) = y_i \tau(x_i)$$

$$\tilde{\mathbb{P}}(y_i) = \exp y_i \tau(x_i)$$

$$\mathbb{P}(y_i) = \frac{\exp y_i \tau(x_i)}{\sum_{k \in \{0,1\}} \exp k \tau(x_i)}$$

$$\mathbb{P}(y_i) = \sigma(\tau(x_i)) \text{ for } y_i = 1$$



FEEDFORWARD NETWORKS

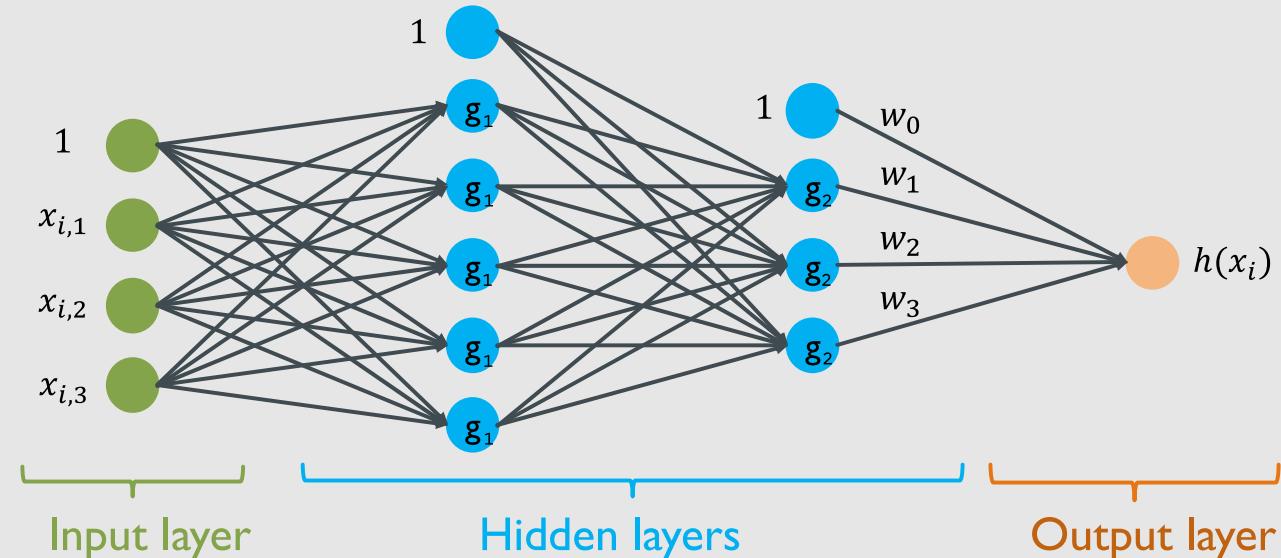
Output: Softmax

$$h(x_i) = S(\underbrace{[w_1, w_2, w_3]^T g_2(x_i) + w_0}_{\tau(x_i)})$$

The output is a vector $\alpha_i \in \{(0,1)\}^K$.

$$\begin{aligned}\log \tilde{\mathbb{P}}(y_i^{(k)}) &= \tau(x_i)^{(k)} \\ \tilde{\mathbb{P}}(y_i) &= \exp \tau(x_i)^{(k)} \\ \mathbb{P}(y_i) &= \frac{\exp \tau(x_i)^{(k)}}{\sum_{l \in \{1, \dots, K\}} \exp \tau(x_i)^{(l)}} \\ \mathbb{P}(y_i) &= S(\tau(x_i))\end{aligned}$$

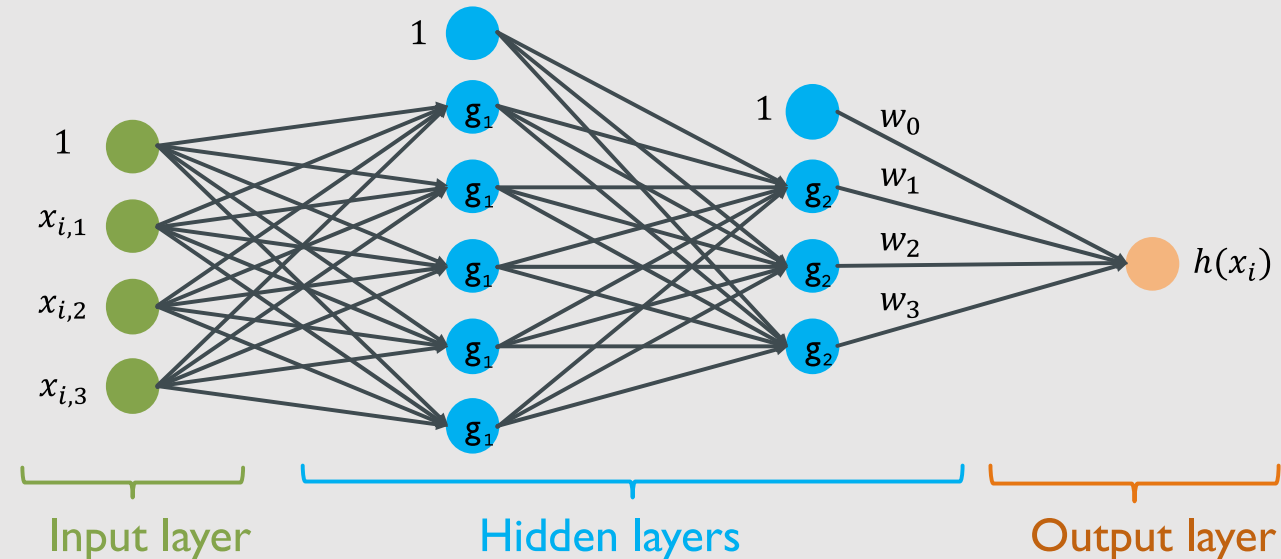
- Classification problems
- $a_i^{(k)} = \mathbb{P}[y_i^{(k)} = 1 | x_i]$
- Negative log-likelihood



FEEDFORWARD NETWORKS

Hidden Layers and Units

- Propagate input to output
- Non-linear input transformations
- Transformations are data driven and are not “seen”
- Hence the term “hidden”



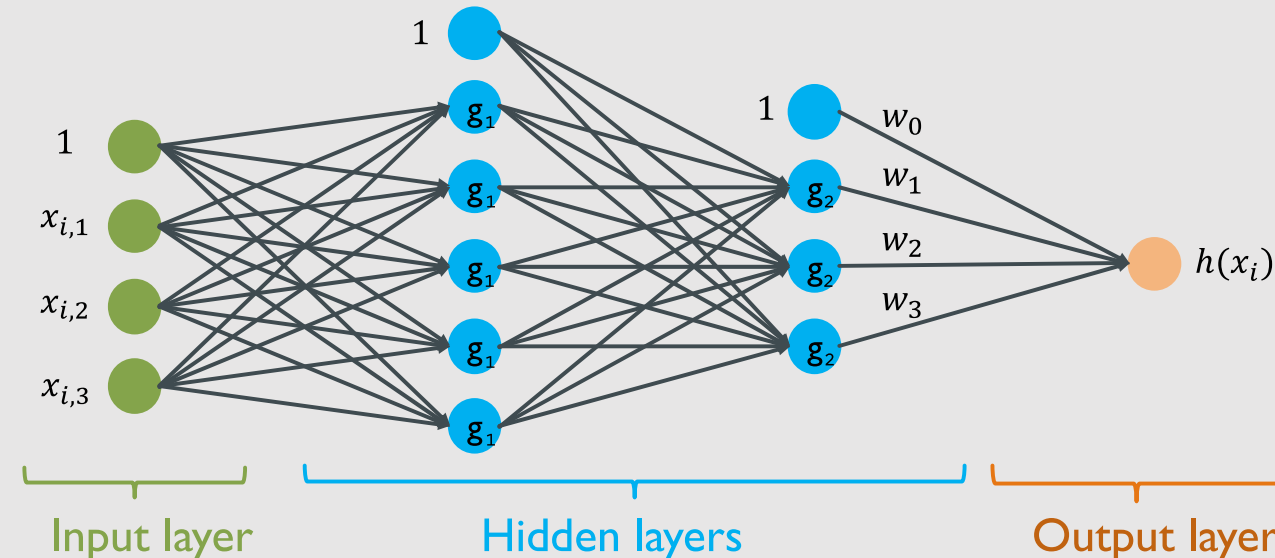
FEEDFORWARD NETWORKS

Hidden Layers and Units

- Propagate input to output
- **Non-linear** input transformations
- Transformations are data driven and are not “seen”
- Hence the term “hidden”

Non-linearities:

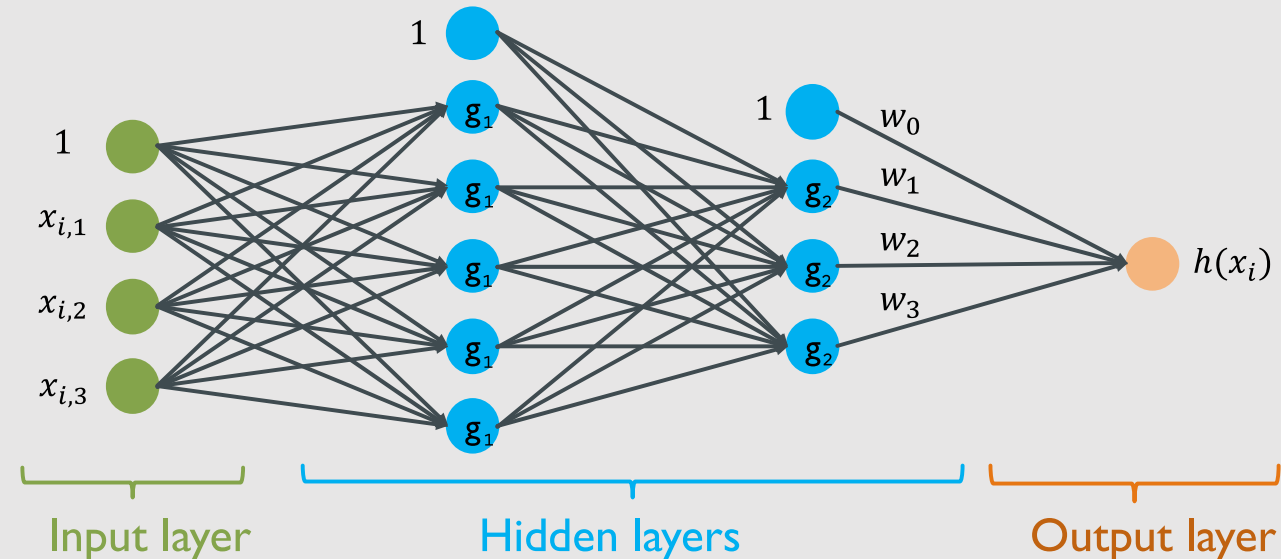
- Logistic and tanh (before deep learning era)
- ReLU variations (during the deep learning era)



FEEDFORWARD NETWORKS

Architecture design

- Chain of layers
 - Number of layers (**depth**)
- Each layer is a set of neurons
 - Number of neurons (**width**)
- Activation functions
- Optimization (algorithm & rate)
- Loss function



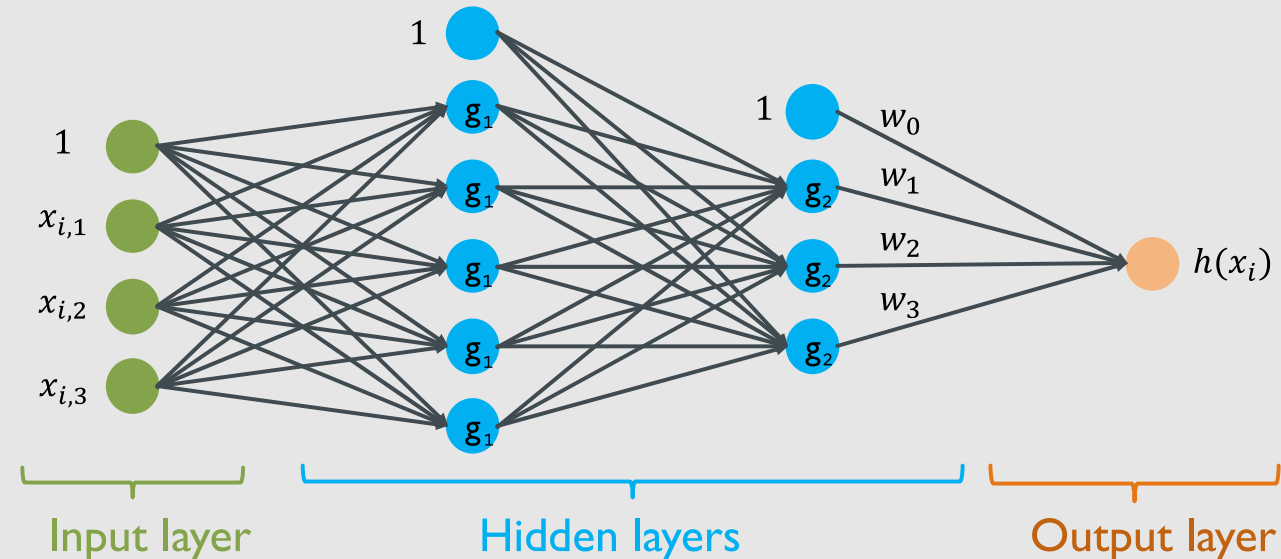
FEEDFORWARD NETWORKS

Architecture design

- Chain of layers
 - Number of layers (**depth**)
- Each layer is a set of neurons
 - Number of neurons (**width**)
- Activation functions
- Optimization (algorithm & rate)
- Loss function

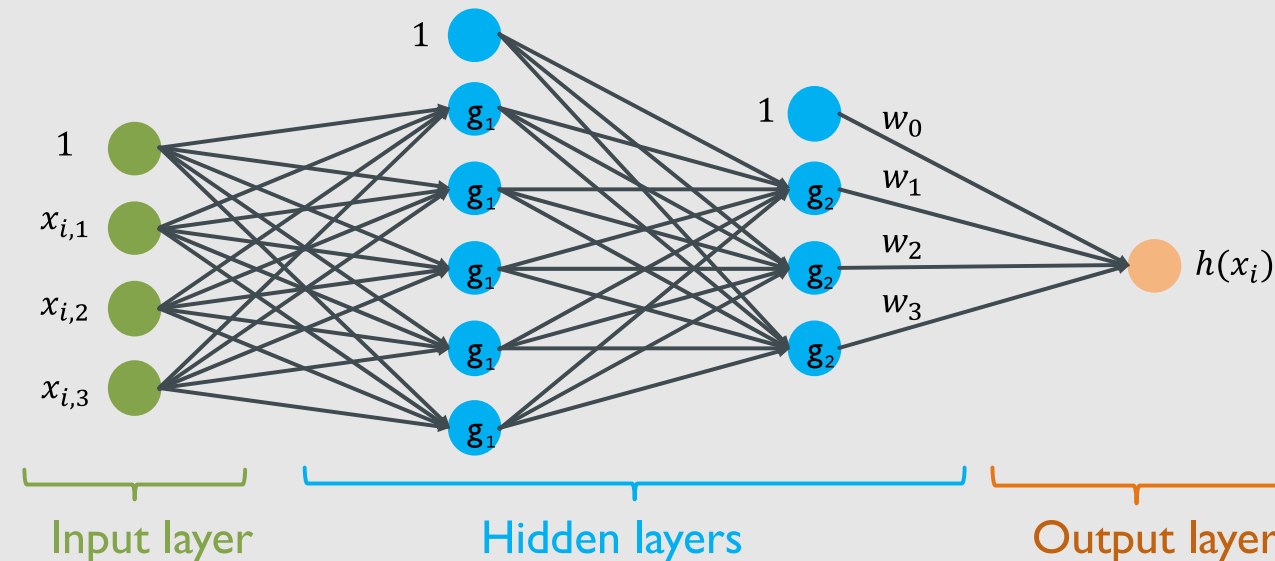
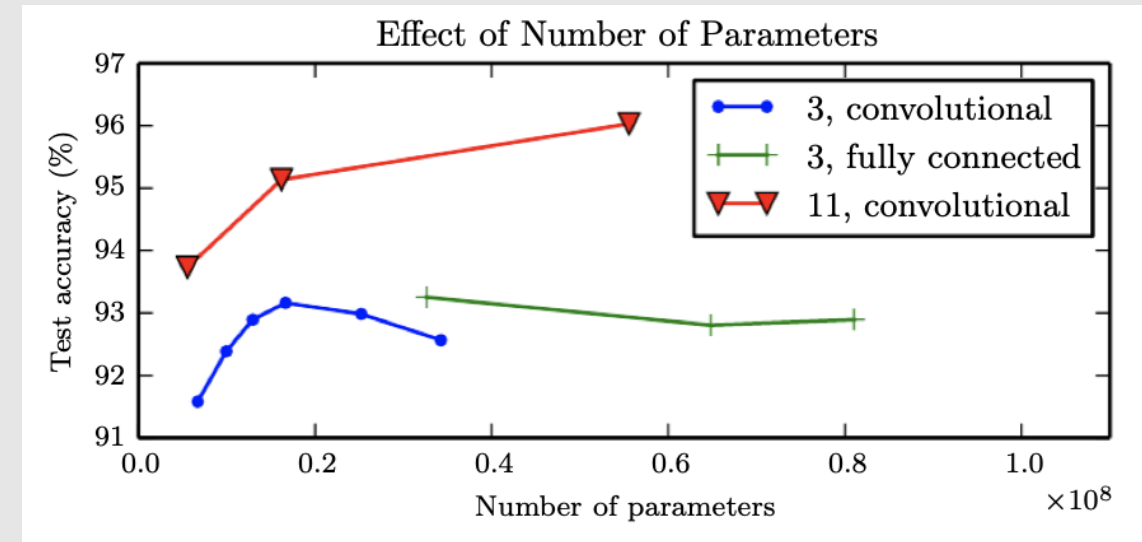
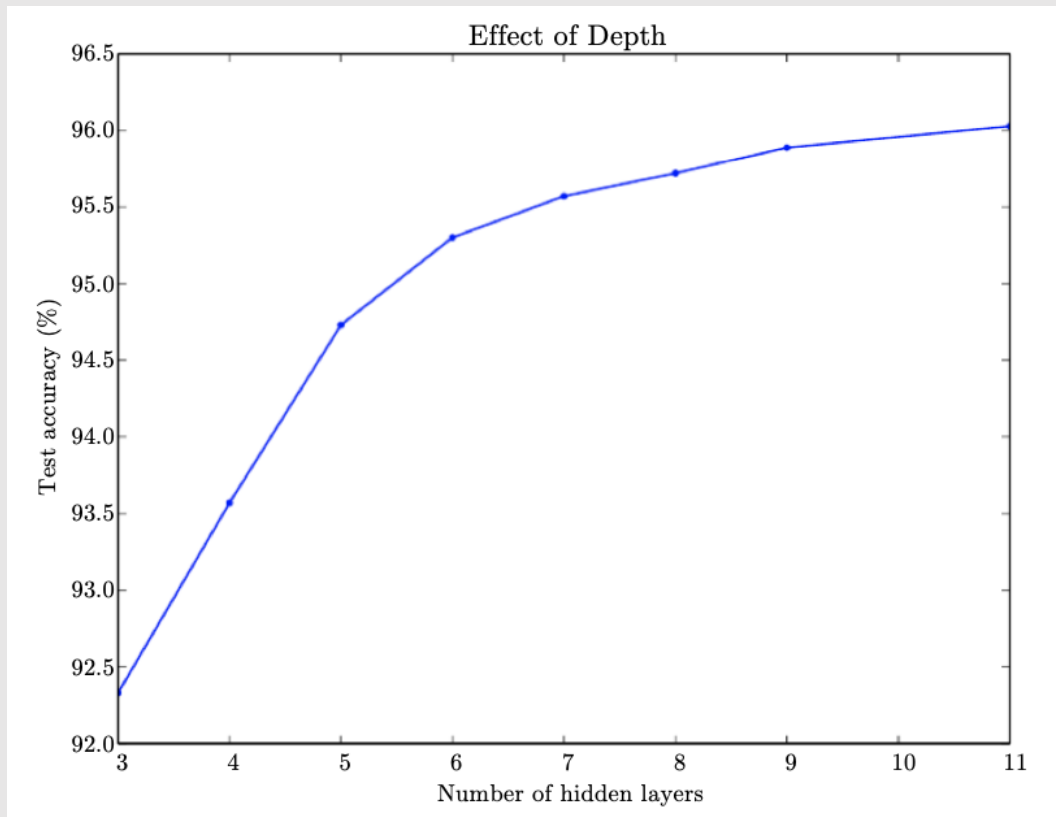
Design choices:

- Empirically
 - trial-and-error
 - Application at hand
- Rules of thumb
 - Overfitting/underfitting



FEEDFORWARD NETWORKS

Architecture design: depth vs width

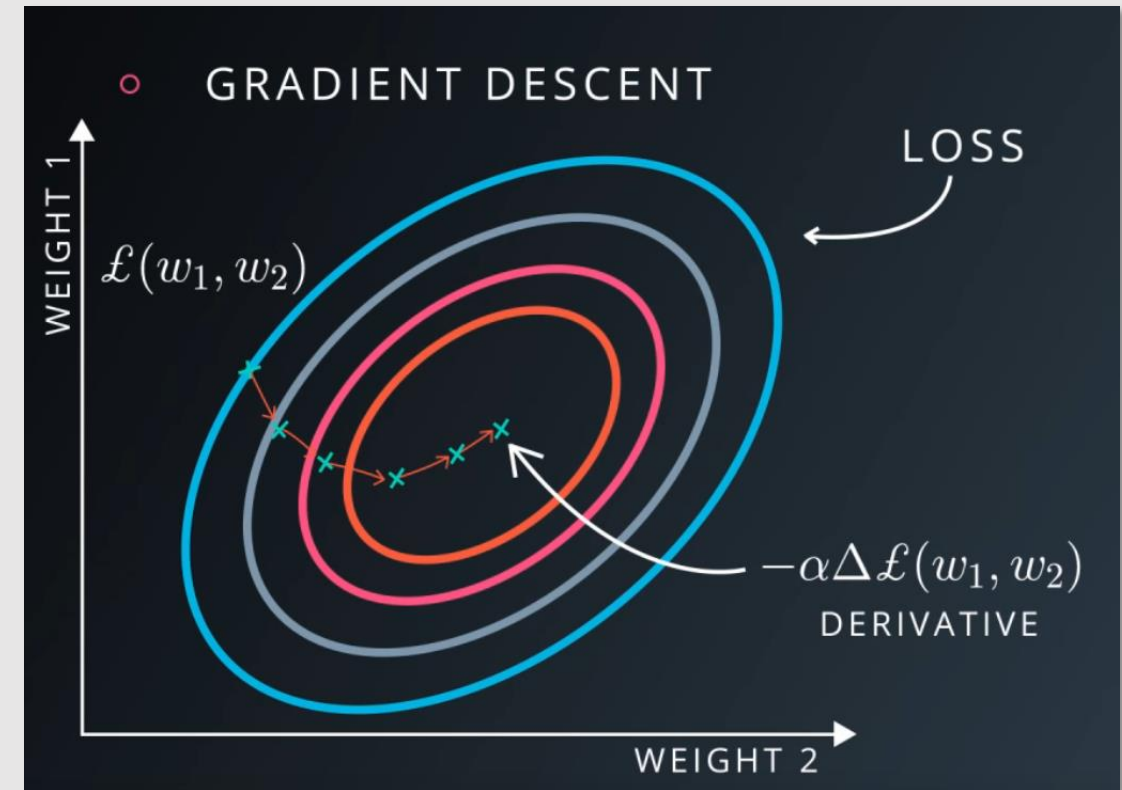


Both images are taken from: Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

FEEDFORWARD NETWORKS

Gradient based optimization

- Randomly initialize weights
- Decide on the loss function $J(w)$
- Back-propagation to compute the gradients of $J(w)$ w.r.t. w
- Move towards the negative direction of the gradient
- Repeat until satisfied



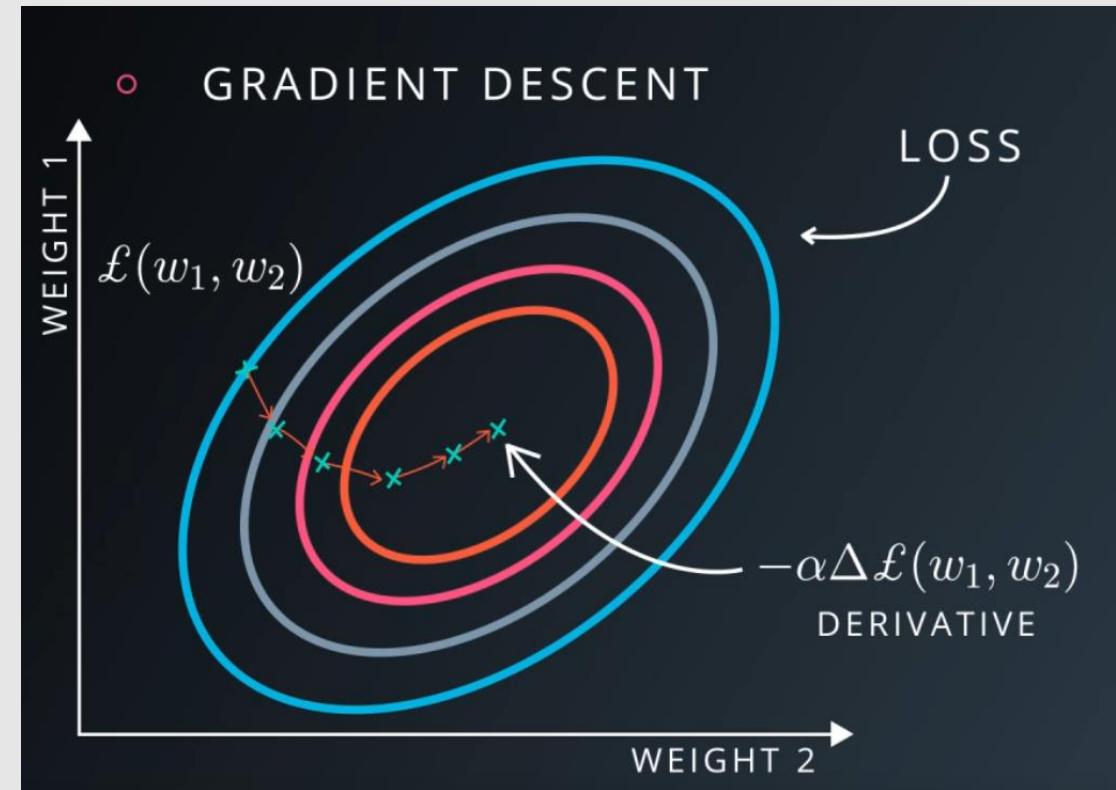
FEEDFORWARD NETWORKS

Gradient based optimization

- Randomly initialize weights
- Decide on the loss function $J(w)$
- Back-propagation to compute the gradients of $J(w)$ w.r.t. w
- Move towards the negative direction of the gradient
- Repeat until satisfied

Complications:

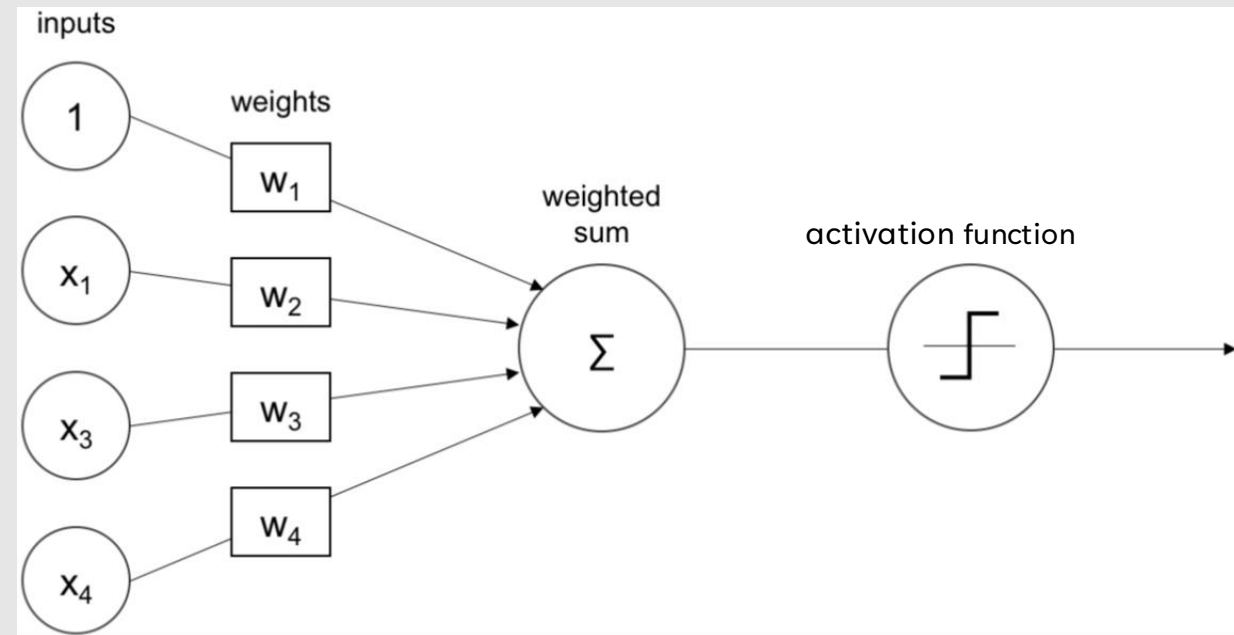
- Local minima



ACTIVATION FUNCTIONS

Determine the output of a neuron

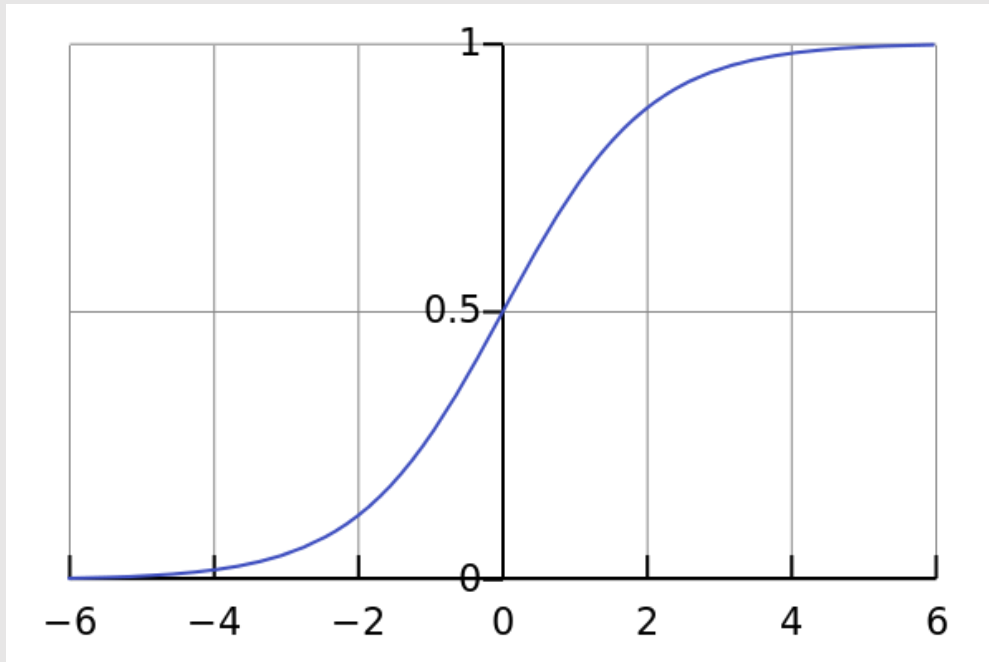
- Indicates when a neuron “fires”
- Nonlinear (why?)
- Affect gradient based optimization algorithm
- Differential and continuous
 - Robust
 - Derivative at every point



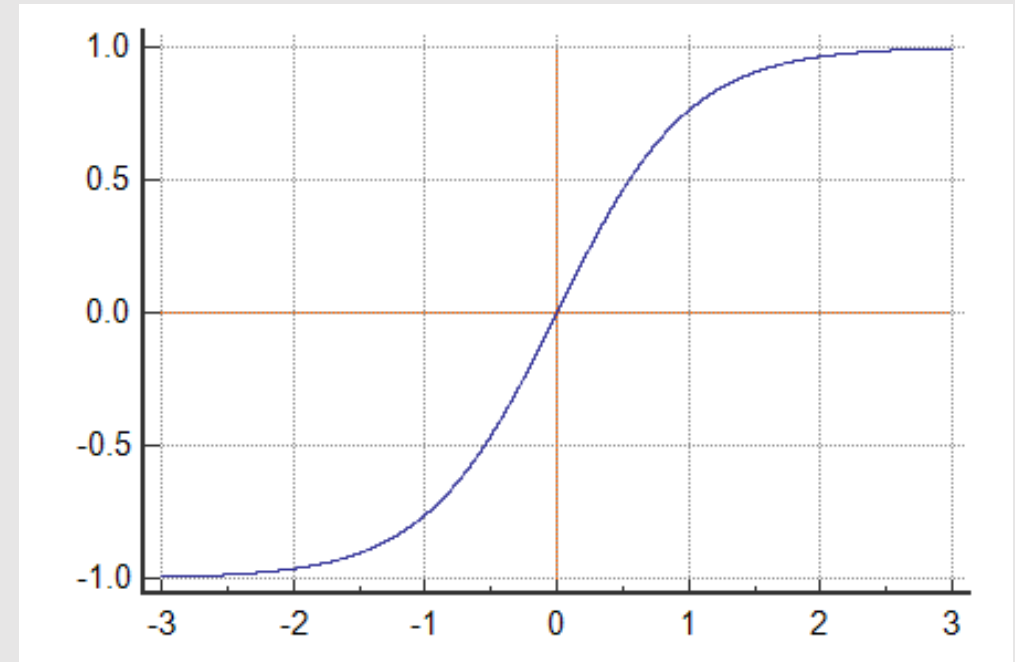
ACTIVATION FUNCTIONS

Sigmoid activations: logistic & tanh

- Logistic: $A = \frac{1}{1+e^{-z}}$



- Tanh: $A = \frac{2}{1+e^{-2z}} - 1$



ACTIVATION FUNCTIONS

Sigmoid activations: logistic & tanh

- Logistic: $A = \frac{1}{1+e^{-z}}$

- Tanh: $A = \frac{2}{1+e^{-2z}} - 1$

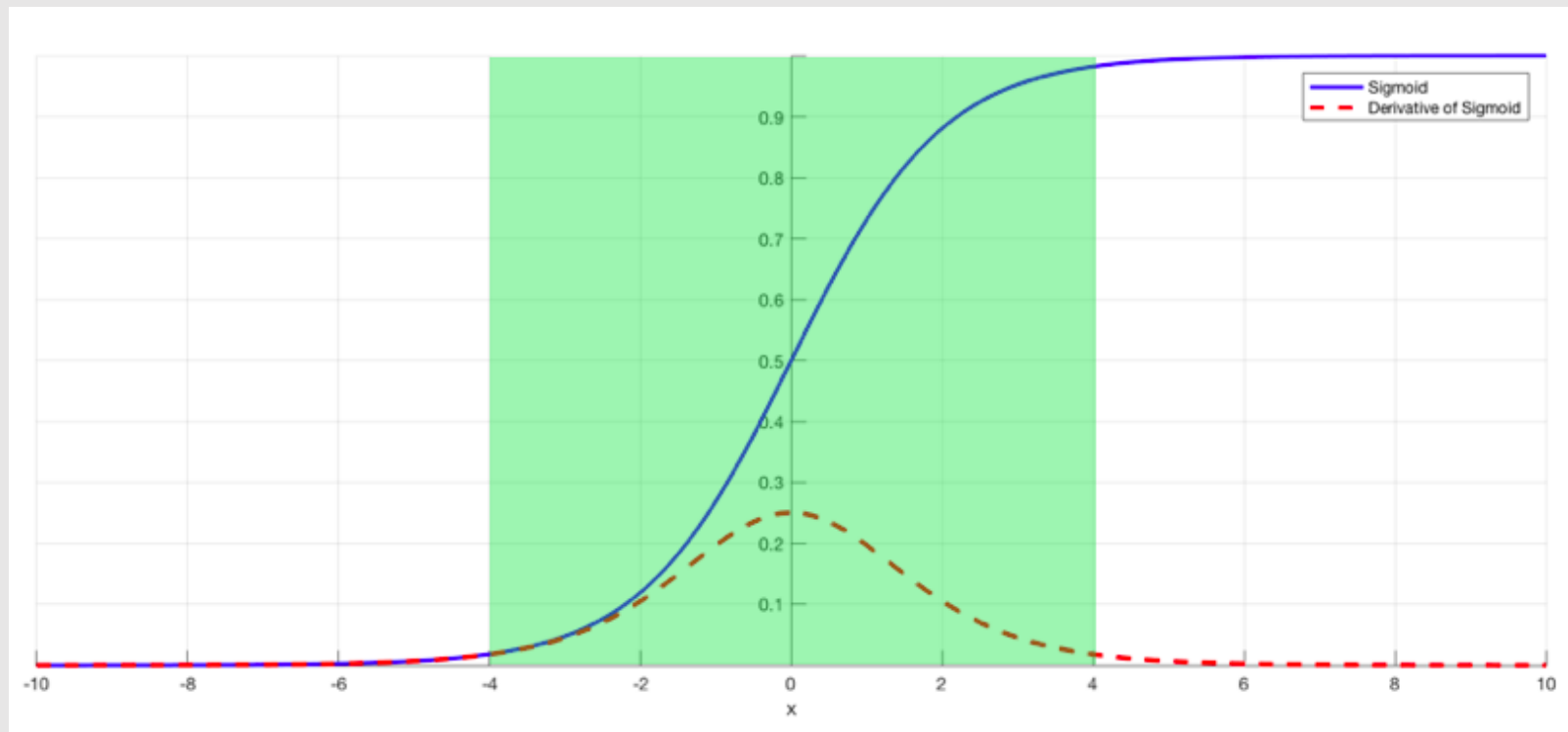
```
def logistic(z) :  
  
    """ Returns the element wise sigmoid  
    function. """  
  
    return 1/(1 + np.exp(-z))
```

```
def tanh(z) :  
  
    """ Returns the element wise  
    hyperbolic tangent function. """  
  
    return np.tanh(z)
```

ACTIVATION FUNCTIONS

Sigmoid activations: logistic & tanh → vanishing gradient

As more layers using certain activation functions are added to neural networks, the gradients of the loss function **approaches zero**, making the network **hard** to train.

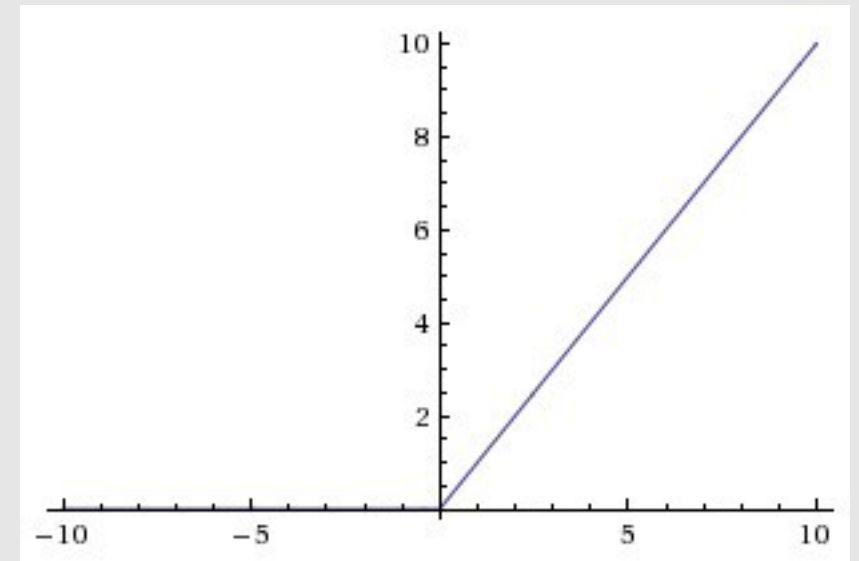


ACTIVATION FUNCTIONS

Rectified Linear Units (ReLU)

- Sigmoids fire all the time, making an ANN heavy.
- ReLU maps input x to $\max(0, x)$
- ReLU lets big numbers pass.
- Faster training: doesn't fire all the time and less computationally expensive.

$$f(z) = \max(0, z)$$

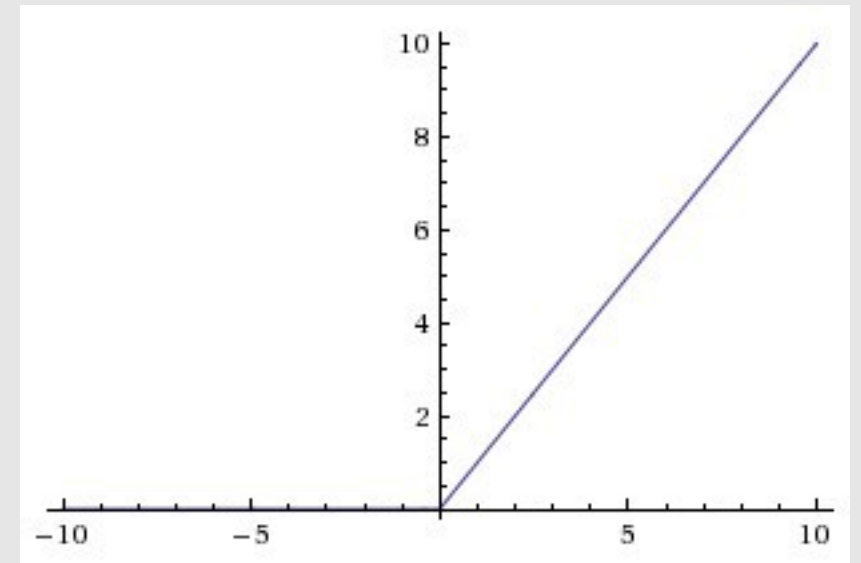


ACTIVATION FUNCTIONS

Rectified Linear Units (ReLU)

```
if input > 0:  
    return input  
else:  
    return 0
```

$$f(z) = \max(0, z)$$



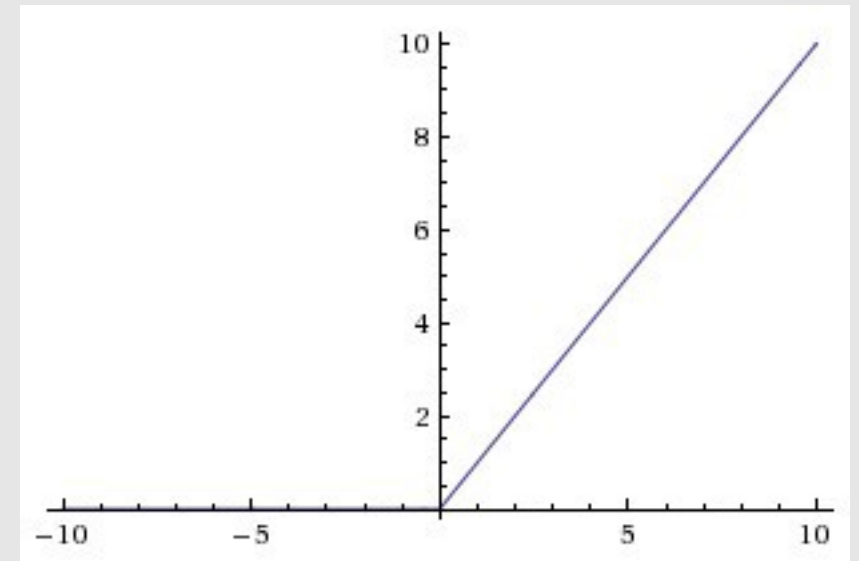
ACTIVATION FUNCTIONS

Rectified Linear Units (ReLU)

```
if input > 0:  
    return input  
else:  
    return 0
```

**Cannot learn via gradient-based methods
on examples for which their activation is
zero.**

$$f(z) = \max(0, z)$$



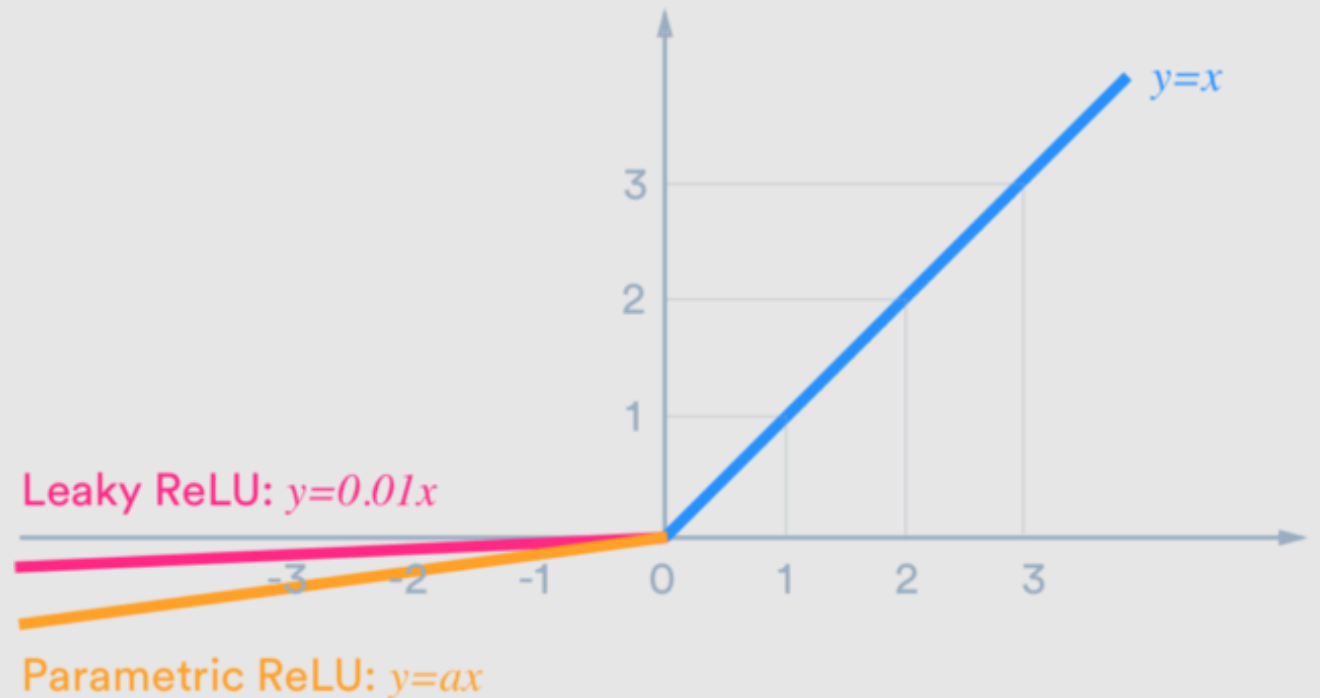
ACTIVATION FUNCTIONS

Leaky RELU

- Allows for a parametric configuration for values of $z < 0$.
- Predefined a value

Parametric ReLU (PReLU)

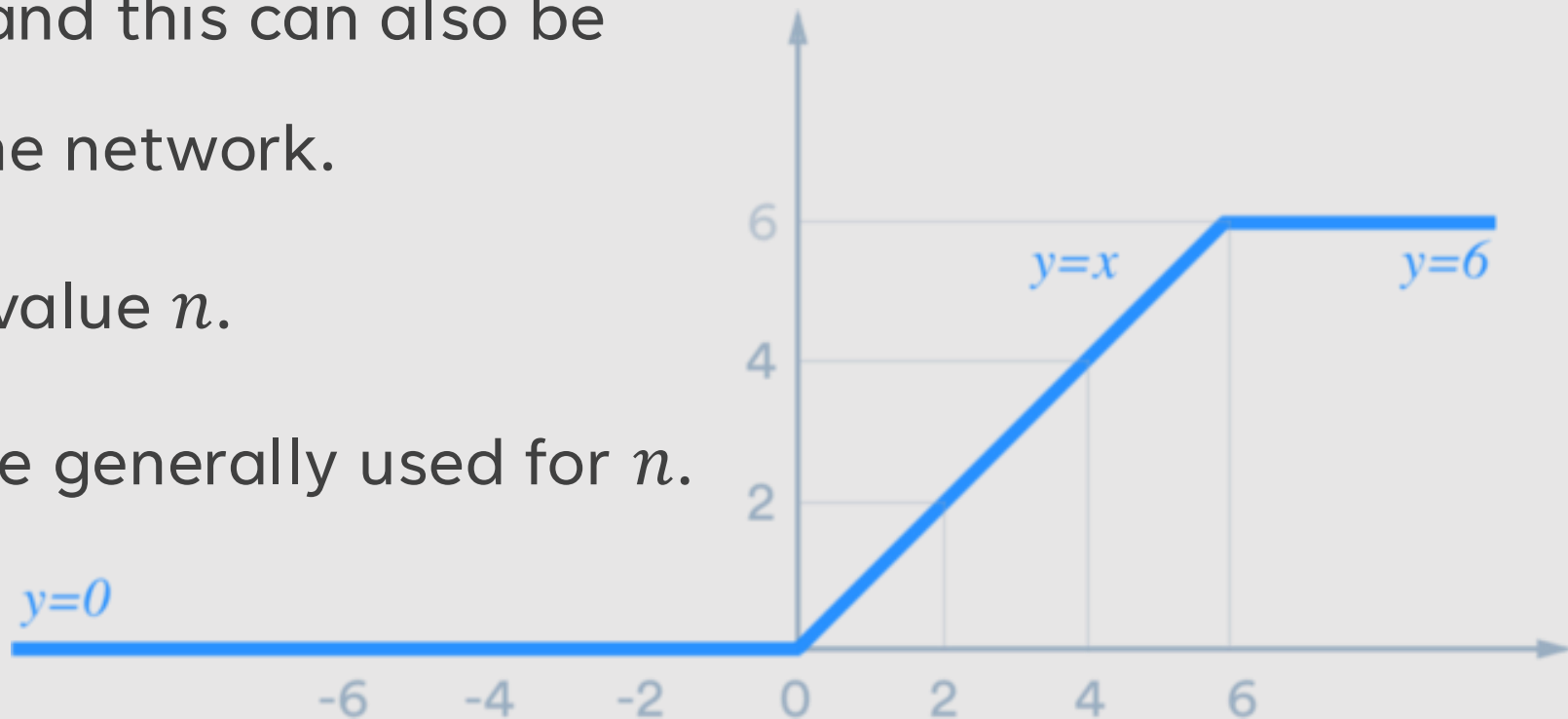
- a is a learnable parameter



ACTIVATION FUNCTIONS

RELU-6

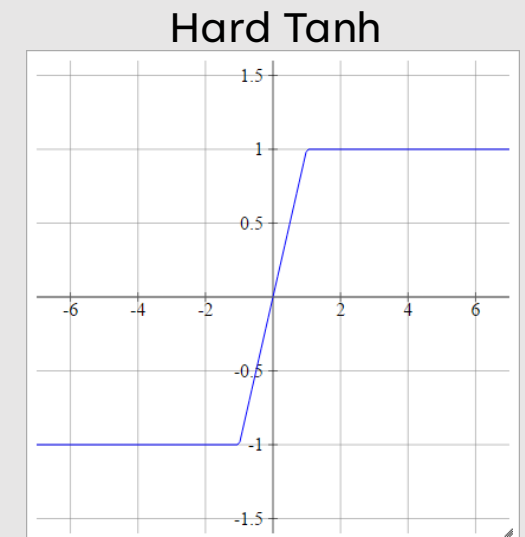
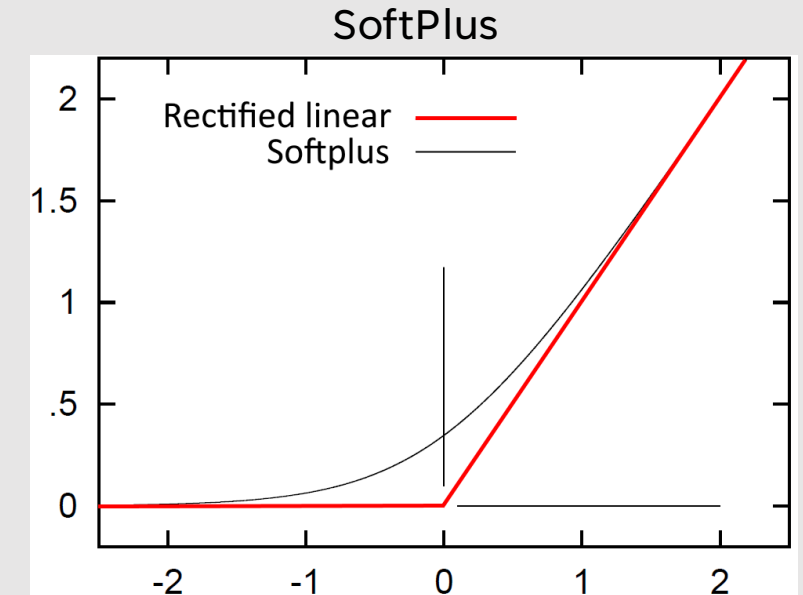
- ReLU has limits $[0, \infty]$ that can also keep pushing activations and this can also be less favourable for the network.
- Solution: Clip z at a value n .
- 6 is an arbitrary value generally used for n .
- Compression



ACTIVATION FUNCTIONS

Other activation functions

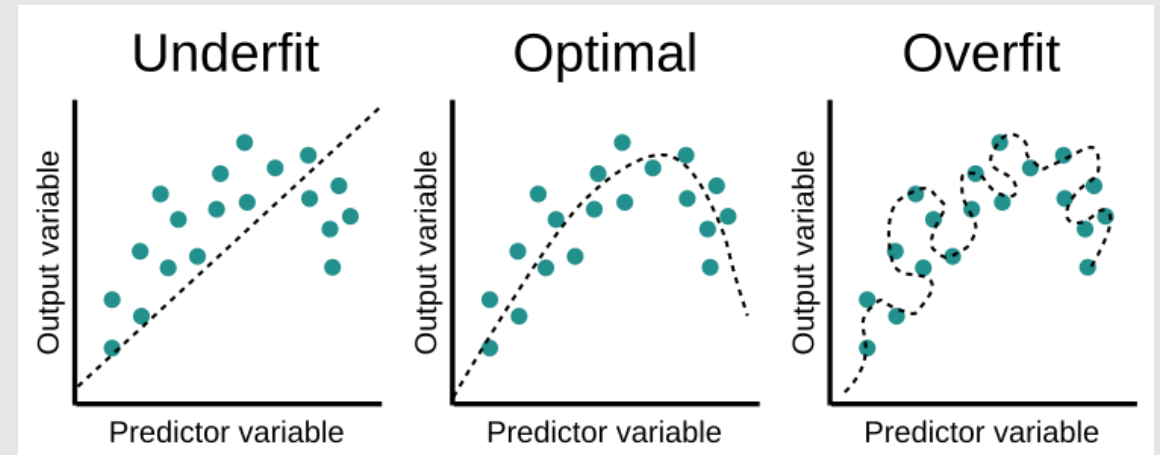
- Radial basis function
- Expensive and difficult to optimise
- Softplus
 - Smooth version of ReLU
 - Empirically underperforms compared to ReLU
- Hard Tanh



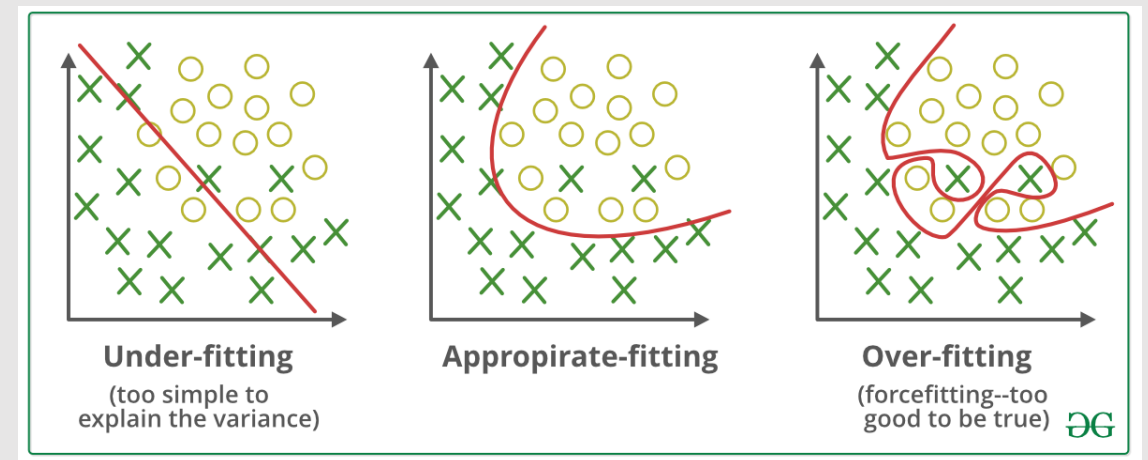
REGULARIZATION

Why do we need regularization?

- Avoid overfitting
- Increase generalization
 - At the expense of increased training error
- One of the major research efforts in the field



<https://www.educative.io/answers/overfitting-and-underfitting>



<https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/>

REGULARIZATION

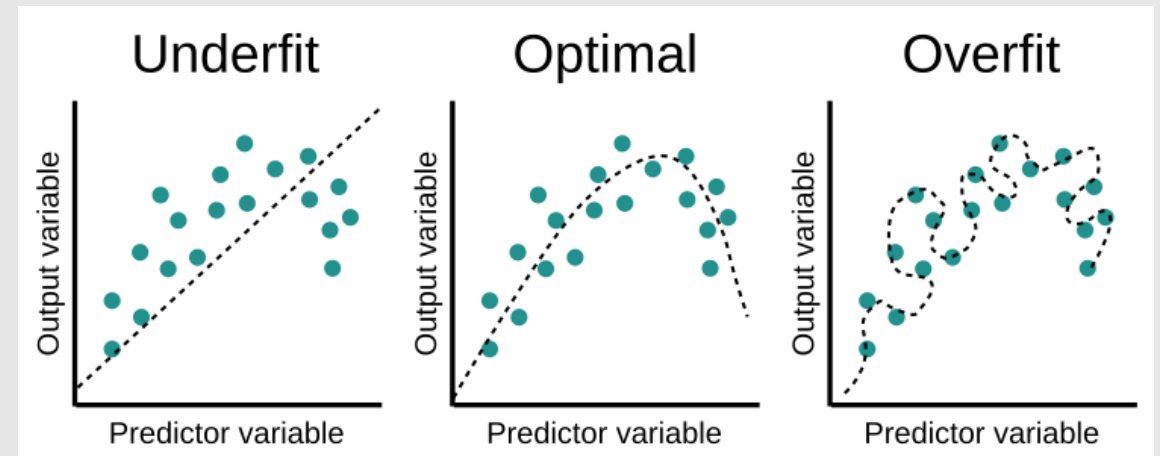
Parameter Norm Penalties

$$\bar{J}(w; X, y) = J(w; X, y) + \alpha\Omega(w)$$

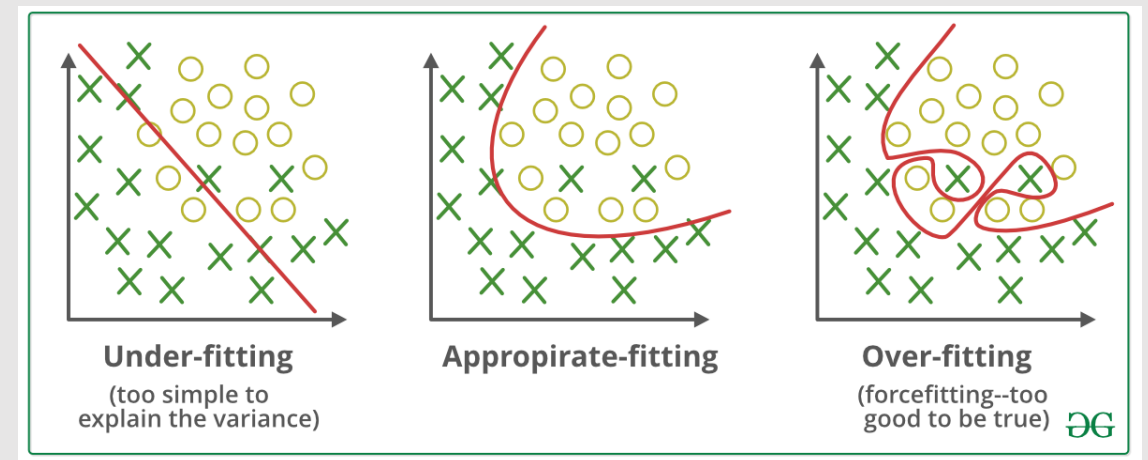
- L^2 parameter regularization

$$\Omega = \frac{w^T w}{2}$$

- Differentiable
- Favours small weights values
- **Very small but not zero**



<https://www.educative.io/answers/overfitting-and-underfitting>



<https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/>

REGULARIZATION

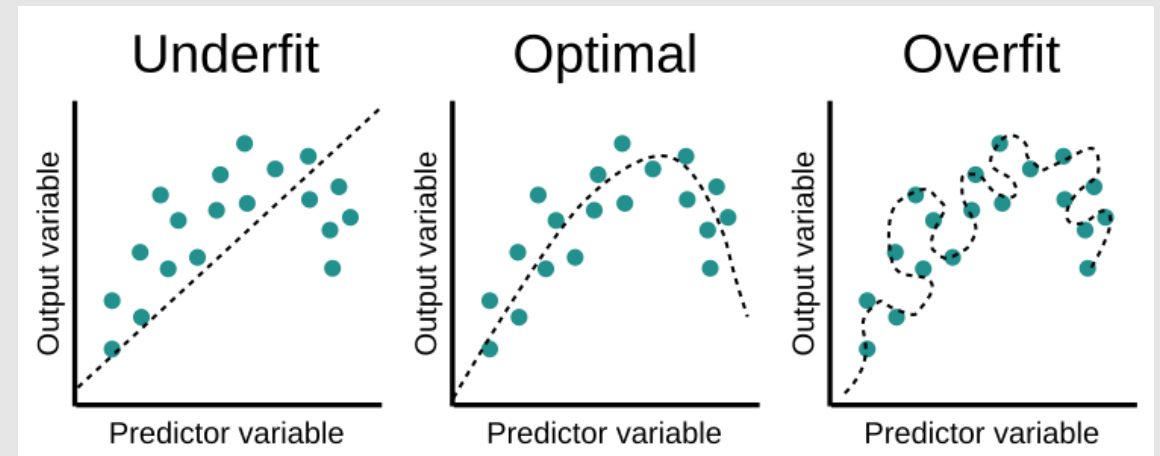
Parameter Norm Penalties

$$\bar{J}(w; X, y) = J(w; X, y) + \alpha\Omega(w)$$

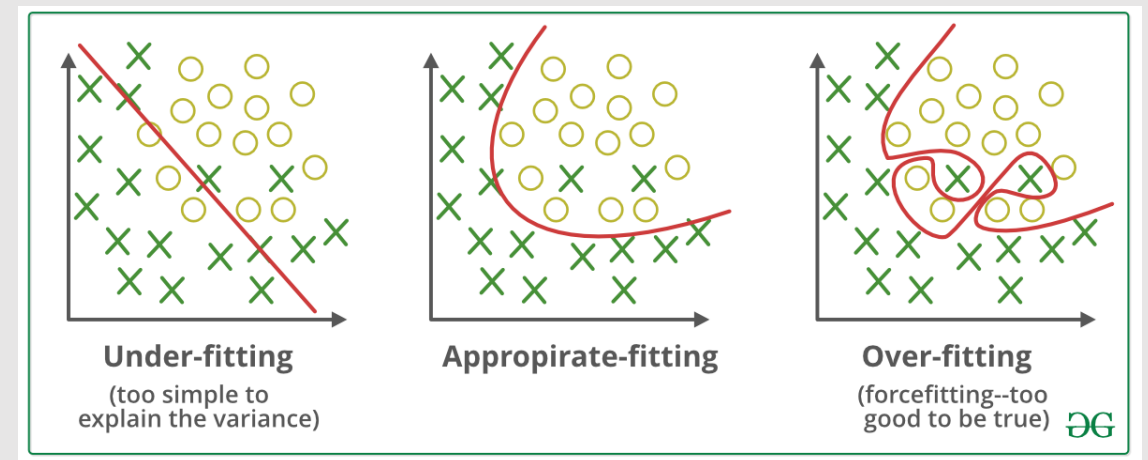
- L^1 parameter regularization

$$\Omega = \sum_i |w_i|$$

- Not everywhere differentiable
- Favours small weights values
- Sets some weights to zero



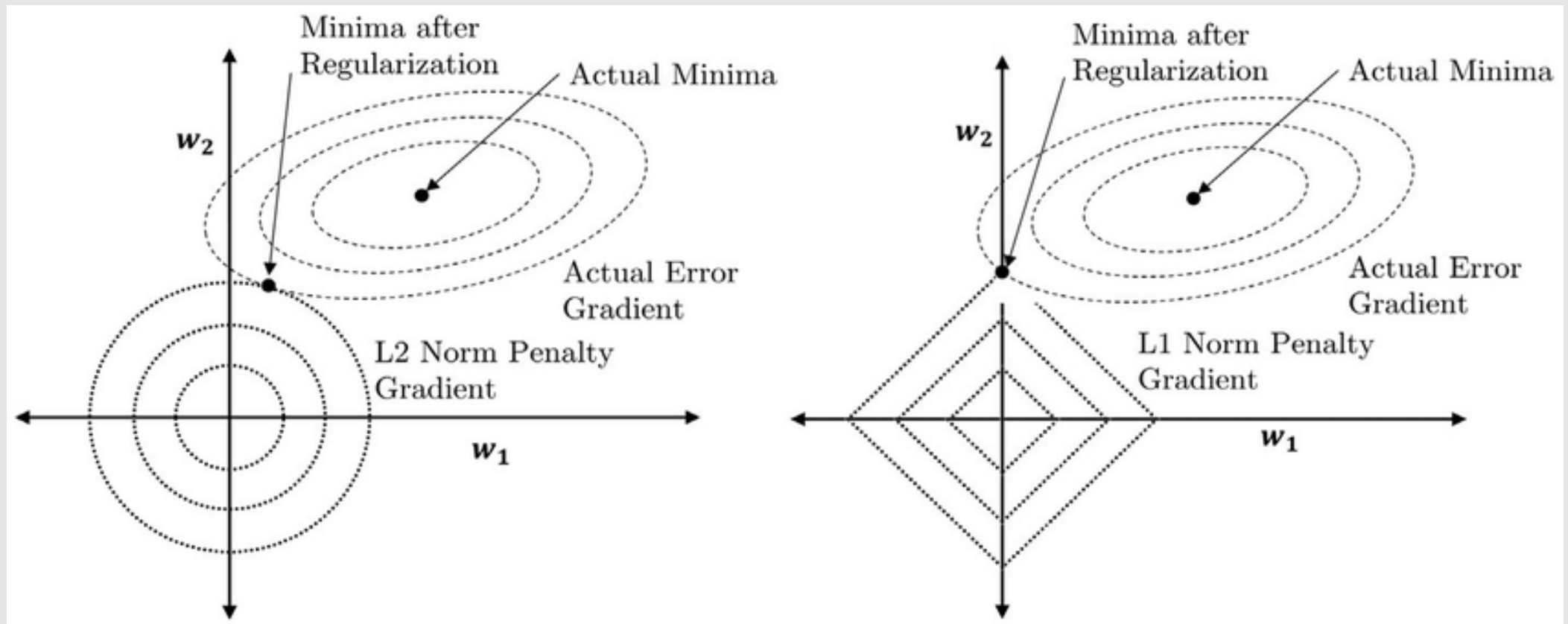
<https://www.educative.io/answers/overfitting-and-underfitting>



<https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/>

REGULARIZATION

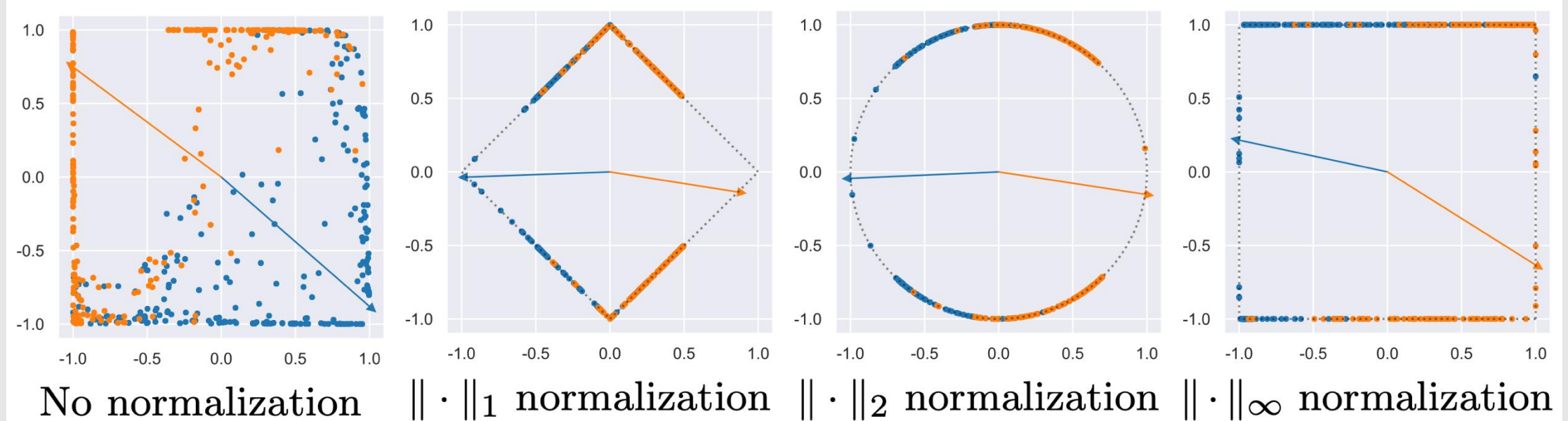
Parameter Norm Penalties



REGULARIZATION

Norm constrained softmax (classification problems)

- Restrict the output of the penultimate layer to lie on a manifold and its magnitude to be equal to a

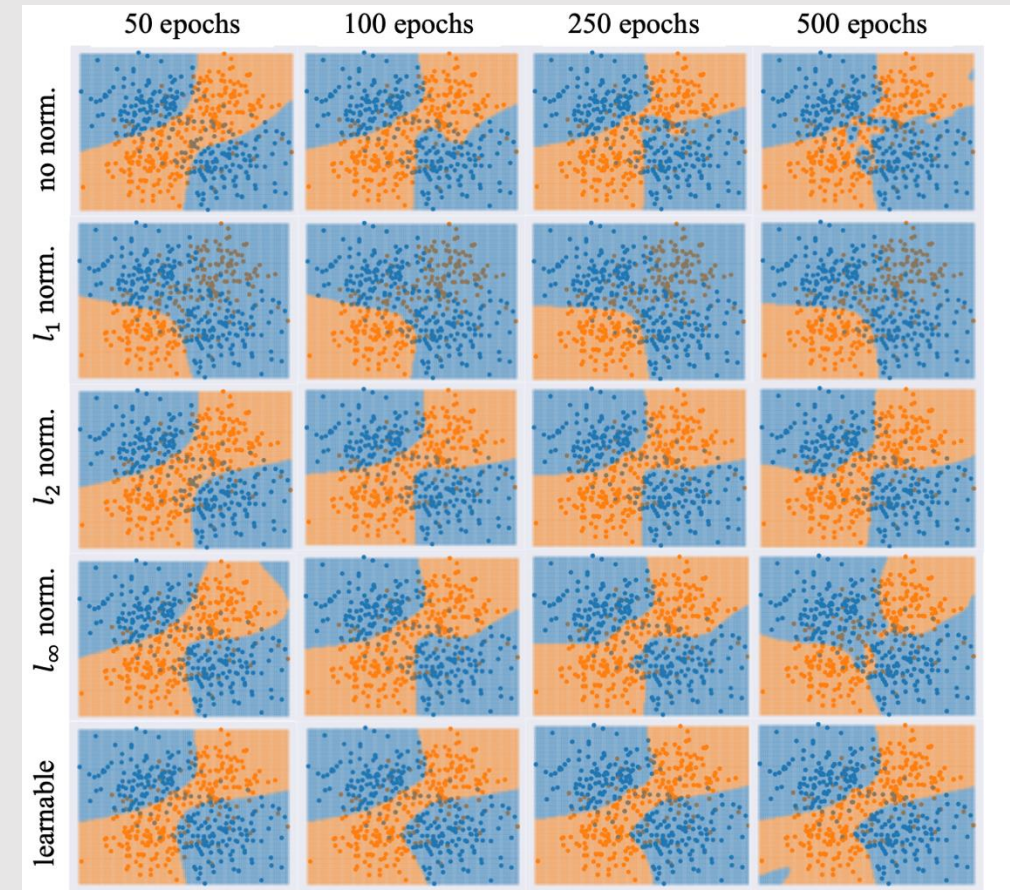
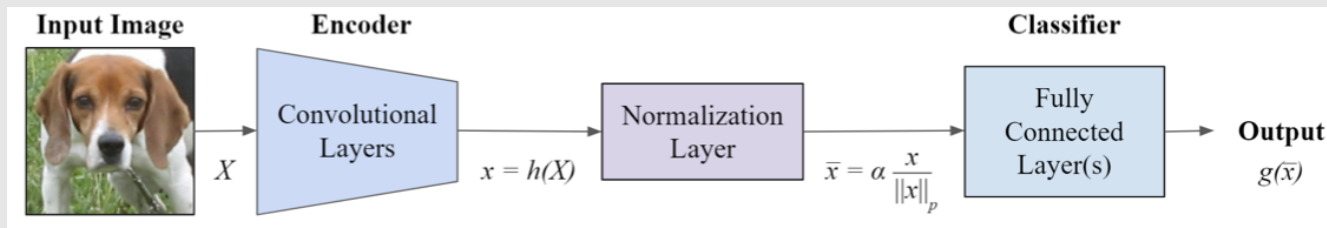


REGULARIZATION

Norm constrained softmax (classification problems)

$$\bar{h}(X) = \alpha \frac{h(X)}{||h(X)||}$$

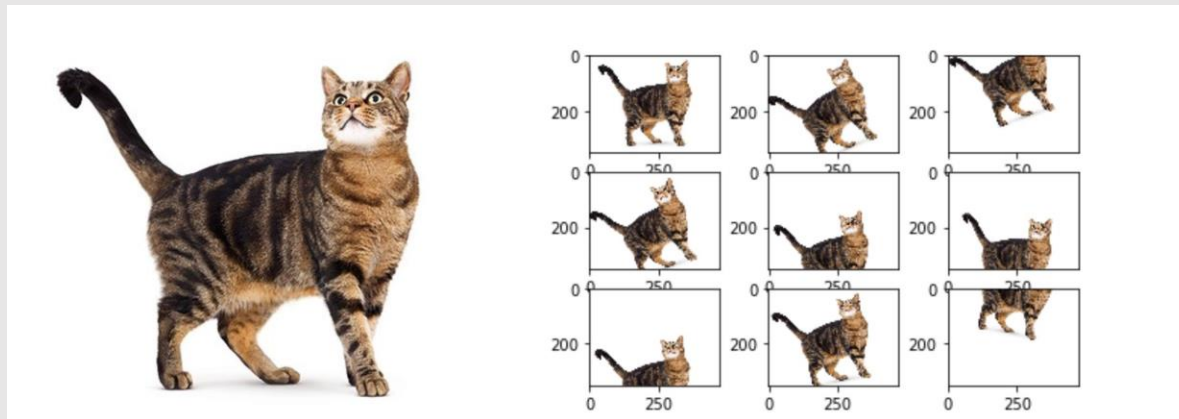
$h(X)$ is the output of the penultimate layer of a network.



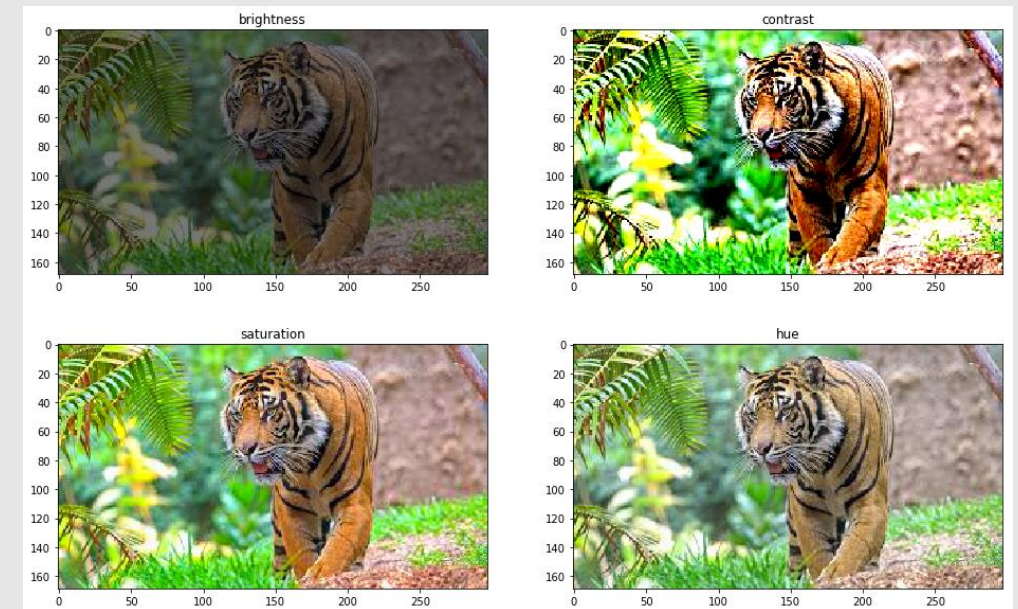
REGULARIZATION

Dataset augmentation

- Generate new data via deterministic transformations
- Inject noise



<https://analyticsindiamag.com/image-data-augmentation-impacts-performance-of-image-classification-with-codes/>



<https://www.v7labs.com/blog/data-augmentation-guide>

REGULARIZATION

Early stopping

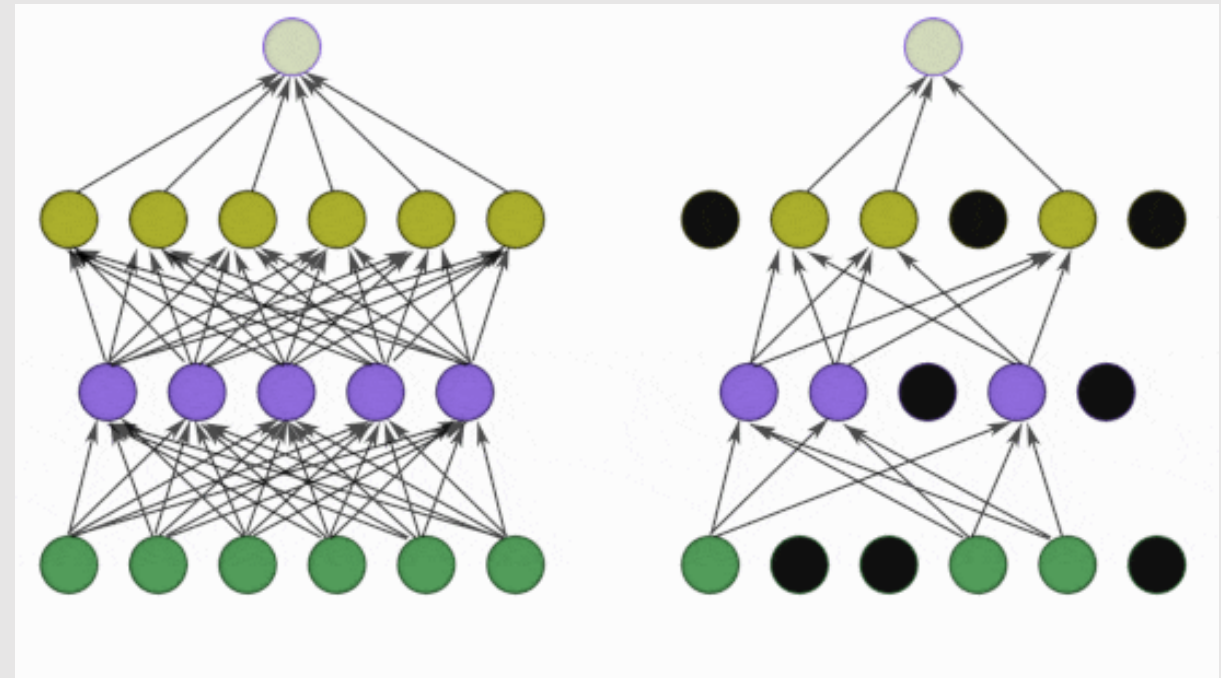
- Stop training when validation loss does not improve
- Sacrifice a subset of the data for validation purposes



REGULARIZATION

Dropout

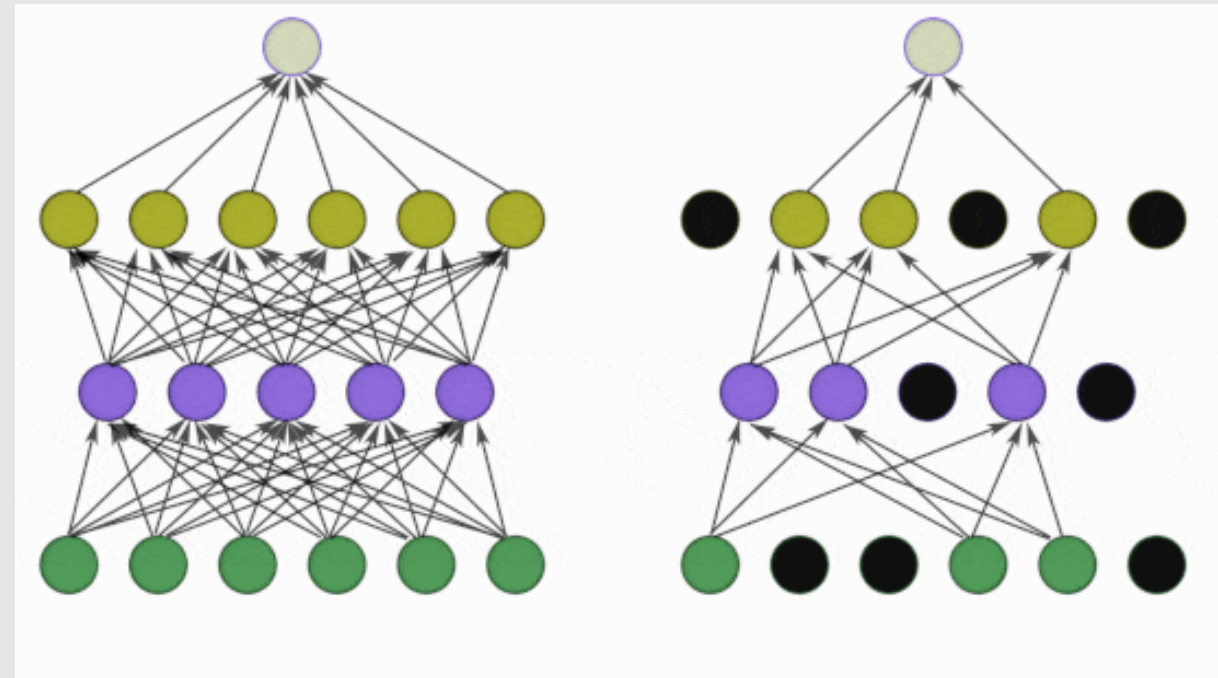
- Inspired by the the brain's plasticity.
- Prevents complex co-adaptation on training data.
- Learn robust features
 - Useful with many different random subsets of the other neurons.



REGULARIZATION

How does dropout work?

- For each training case, a percentage of hidden neurons are randomly selected and set to zero and this leaves different architectures for each case.
- Dropout roughly doubles the number of iterations required to converge. However, training time for each epoch is less.



TRAINING DEEP NEURAL NETWORKS

Deep neural networks are **data hungry**...

- Millions of trainable parameters
- Huge datasets

TRAINING DEEP NEURAL NETWORKS

Deep neural networks are **data hungry**...

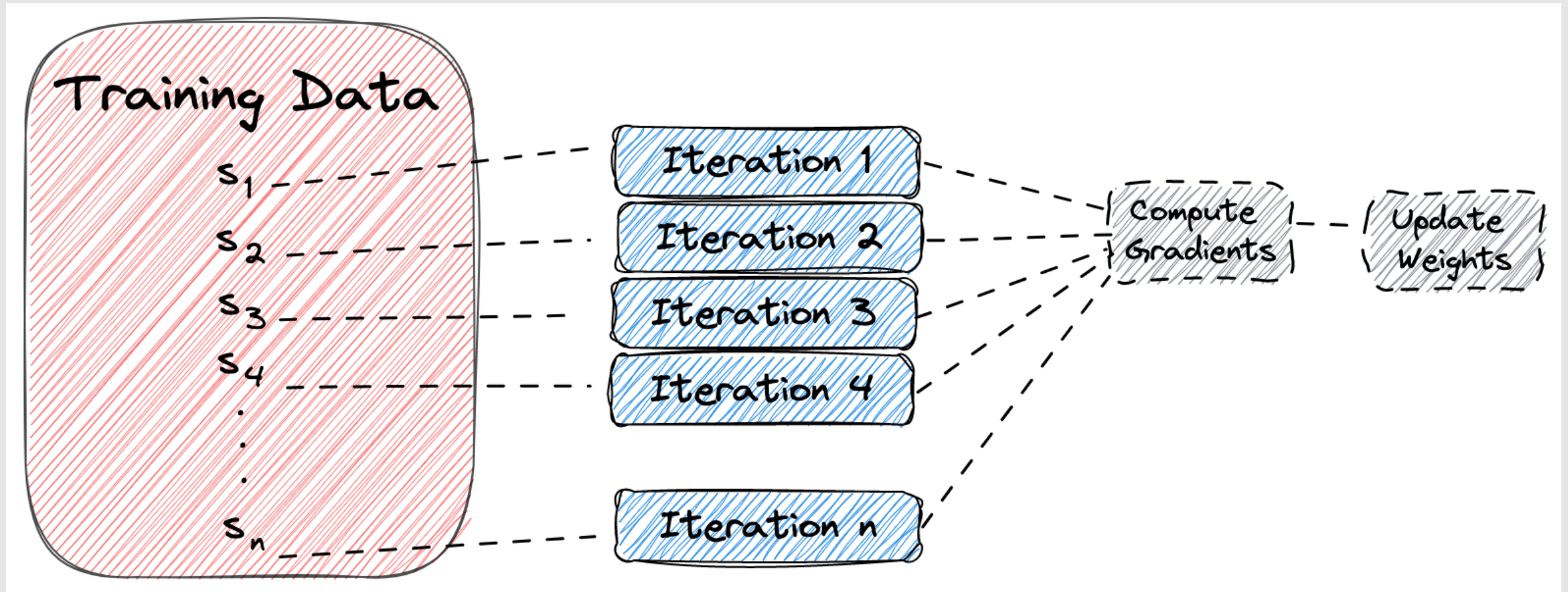
- Millions of trainable parameters
- Huge datasets

How can we handle datasets with millions of examples?

- We cannot load the dataset into the memory of a computer

TRAINING DEEP NEURAL NETWORKS

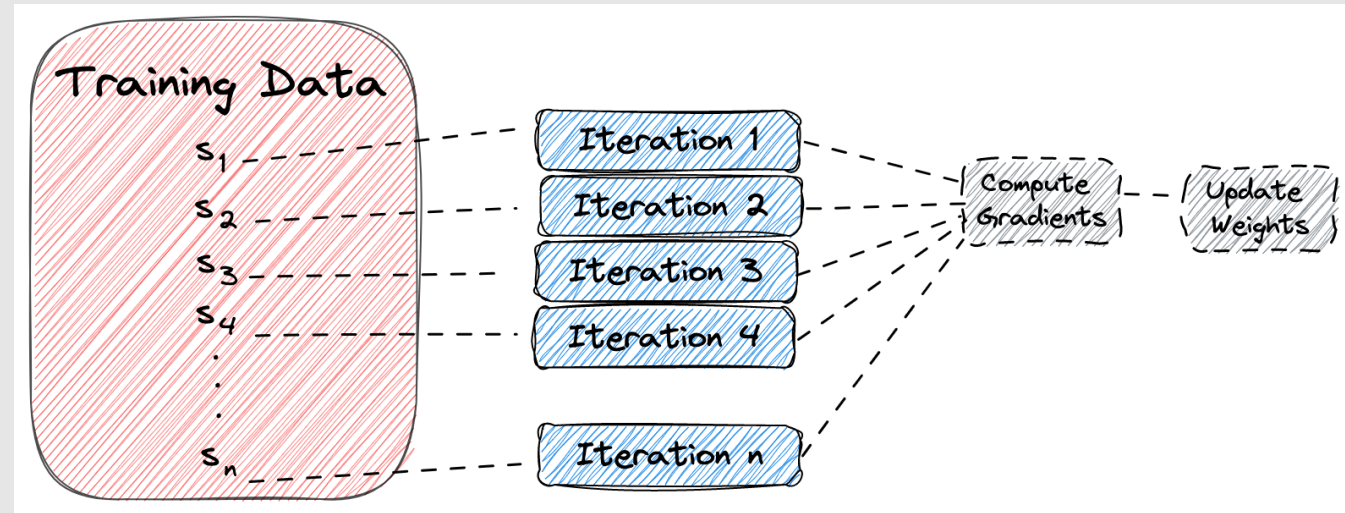
Minibatch training



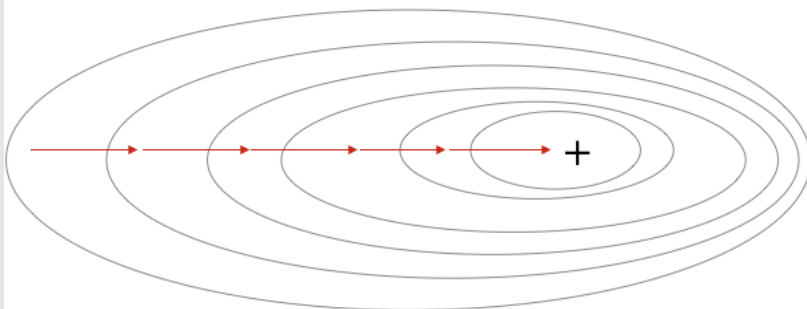
TRAINING DEEP NEURAL NETWORKS

Minibatch training

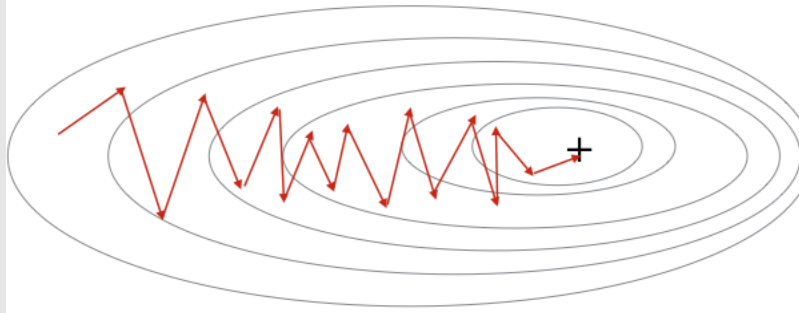
- “Avoid” local minima
- Memory efficient
- Parallelization



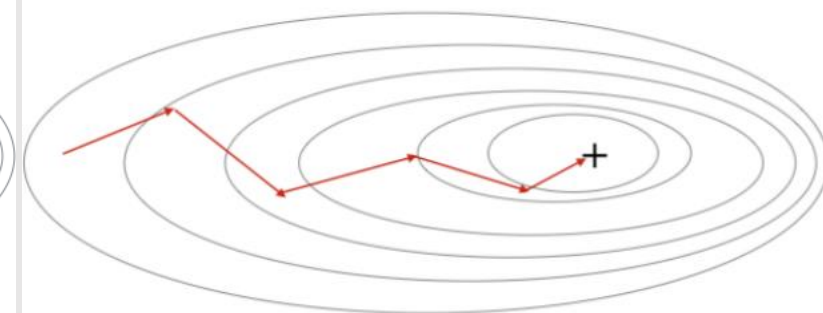
Gradient Descent



Stochastic Gradient Descent



Mini-Batch Gradient Descent



TRAINING DEEP NEURAL NETWORKS

Challenges in training deep neural networks

- **Local Minima**
 - Non-convex, highly nonlinear functions
 - Affect the quality of the solution

TRAINING DEEP NEURAL NETWORKS

Challenges in training deep neural networks

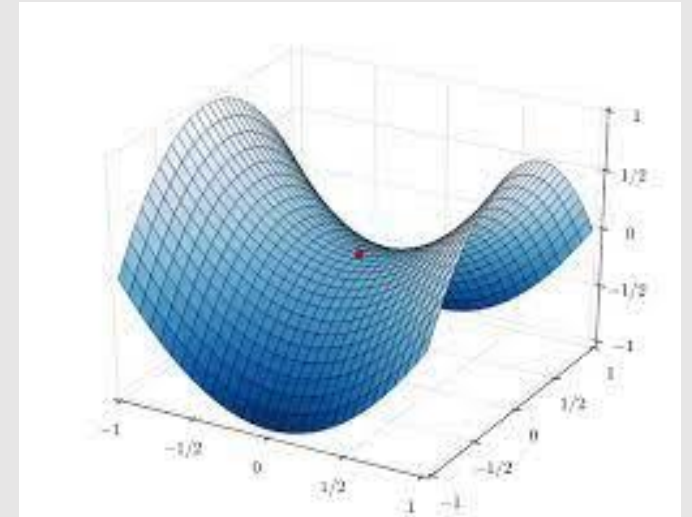
- **Local Minima**
 - Non-convex, highly nonlinear functions
 - Affect the quality of the solution

Practically it is not a problem to end up with a local minimum if the cost function that corresponds to it is very low.

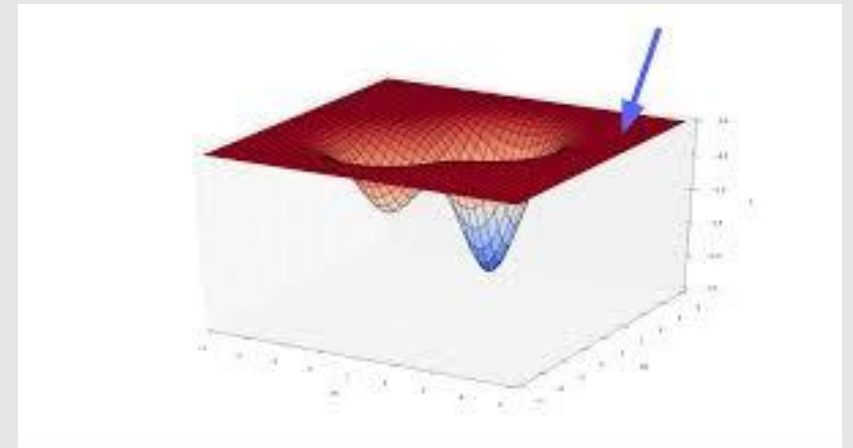
TRAINING DEEP NEURAL NETWORKS

Challenges in training deep neural networks

- Flat regions and saddle points
 - Converge to no local minima points
 - Affect the quality of the solution
- Optimization algorithm parameterization



https://en.wikipedia.org/wiki/Saddle_point



https://www.cs.toronto.edu/~rgrosse/courses/csc321_2017/slides/lec7.pdf

TRAINING DEEP NEURAL NETWORKS

Challenges in training deep neural networks

- **Parameter initialization**
 - Defined by the user
 - Determines whether the algorithms converges at all
 - Affect the quality of the final solution (high vs low cost)
 - Strategies are simple and heuristic

TRAINING DEEP NEURAL NETWORKS

Challenges in training deep neural networks

- **Parameter initialization guidelines**
 - Initial parameters need to “break symmetry”
 - Neighbouring neurons must have different initial values
 - Deterministic methods (cost, optimization) → same initialization - same updates
 - Reduces networks capacity to learn interesting patterns

TRAINING DEEP NEURAL NETWORKS

Challenges in training deep neural networks

- **Parameter initialization guidelines**
 - Initial parameters with values drawn randomly from a **Gaussian** distribution
 - Larger values yield a stronger symmetry breaking effect
 - Too large values result in exploding values during forward of back propagation

TRAINING DEEP NEURAL NETWORKS

Challenges in training deep neural networks

- **Parameter initialization guidelines**
 - **Glorot** initialization
 - Consider a layer with m inputs and n outputs

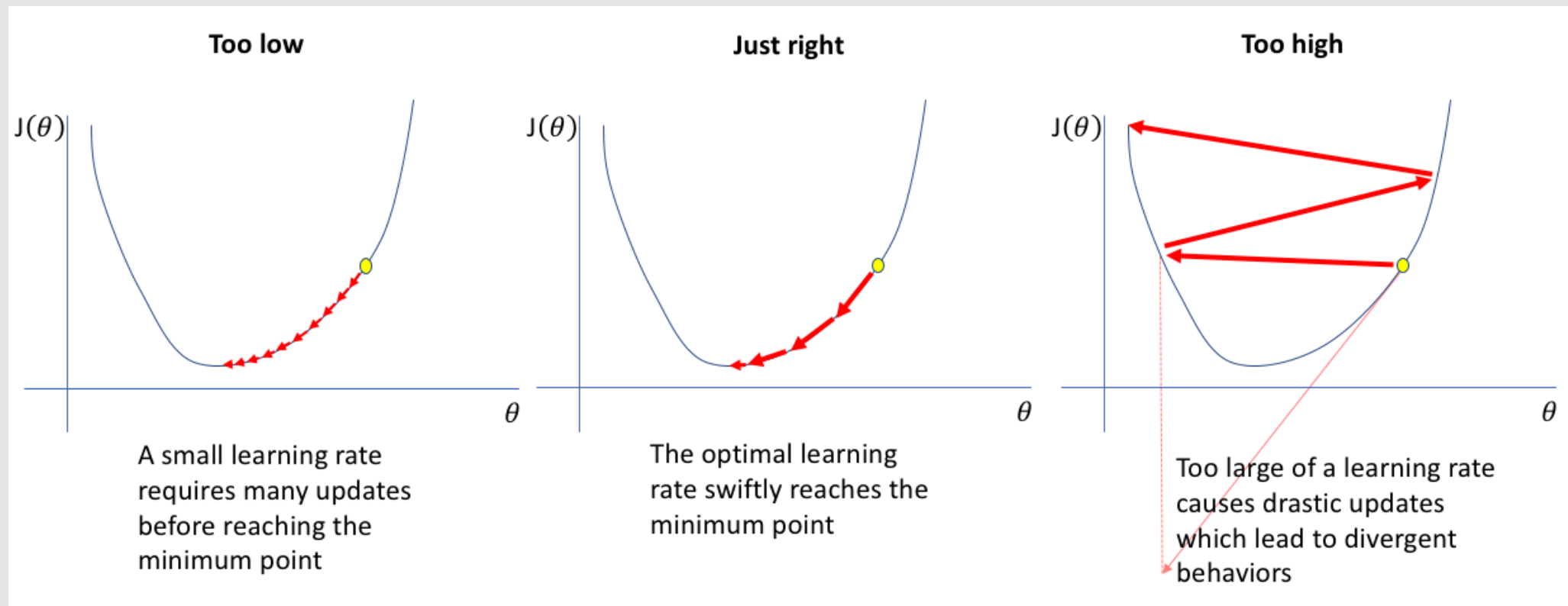
$$W \sim U \left(-\frac{\sqrt{6}}{\sqrt{m+n}}, \frac{\sqrt{6}}{\sqrt{m+n}} \right)$$

- All layers have the same activation variance

TRAINING DEEP NEURAL NETWORKS

Challenges in training deep neural networks

- Learning rate of the gradient-based optimization algorithm



TRAINING DEEP NEURAL NETWORKS

Challenges in training deep neural networks

- **Learning rate of the gradient-based optimization algorithm**
 - Adaptive learning rate
 - Reduce learning rate as the training progresses
 - Algorithms with adaptive learning rates
 - AdaGrad
 - RMSProp
 - Etc.

TRAINING DEEP NEURAL NETWORKS

Challenges in training deep neural networks

- **Meta-learning - Batch normalization**
 - Increase the stability and performance of neural network training
 - This normalizes the output from a layer with zero mean and a standard deviation of 1.
 - This reduces overfitting and makes the network train faster. It is very useful in training complex neural networks.

TRAINING DEEP NEURAL NETWORKS

Challenges in training deep neural networks

- Meta-learning - Unsupervised pretraining
 - Restricts the search space to “something” meaningful

https://www.researchgate.net/figure/Stacked-autoencoders-based-deep-learning-Unsupervised-pre-training-on-the-left_fig2_326501953

